TABLE OF CONTENTS

1. PROBLEM STATEMENT AND APPROACH

1.1 Problem Statement

The UIDAI ecosystem handles massive volumes of Aadhaar enrollment and update data across diverse geographical regions. However, manual auditing of these datasets to identify systemic inefficiencies, regional disparities, and temporal anomalies is slow and prone to oversight. As of January 2026, there is a critical need for an automated, scalable solution that can synthesize complex statistical distributions and visual trends into actionable administrative insights.

1.2 Our Approach

Our team developed a Full Self-Made Analytical Pipeline that automates the entire audit lifecycle. Unlike standard analysis, our pipeline integrates:

> End-to-End Automation: A seamless data-to-report flow that ingests raw Aadhaar datasets and outputs a professional, audited PDF. Research-Grade Statistics: Implementation of advanced mathematical frameworks to identify anomalies beyond basic averages. Deep AI Synthesis: Utilizing multimodal LLMs to perform "deep-dive" analysis on visual data, translating complex charts into policy-ready text.

1.3 Key Innovation: The Autonomous AI-Auditor

The core innovation is our Proprietary Multimodal Pipeline. We moved beyond static dashboards by creating a system that "thinks" like an auditor. By feeding our professional-grade visualizations directly into research-tuned AI models, we extract "deep analysis" insights—such as identifying specific district-level bottlenecks or temporal surges—that traditional rule-based systems would miss. This represents a shift from descriptive analytics to prescriptive governance

2. DATASETS USED

2.1 Dataset Overview

The pipeline ingests large-scale Aadhaar transaction datasets containing over X million records (placeholder for your specific row count) across 40 States and Union Territories. The data encompasses three core service categories:

a) Enrolment: Records of new Aadhaar generation.

b) Demographic: Updates to personal details such as address, name, or gender.

c) Biometric: Updates to iris, fingerprint, or facial photographs.

2.2 Data Structure

Our pipeline is designed to handle high-dimensional time-series data. The primary features analyzed include:

a) Temporal Features: Date and month of transaction, allowing for the identification of seasonality and "system collapse" events.

b) Geographical Features: State and District identifiers to map regional performance disparities.

c) Performance Metrics: Total counts of successful generations vs. rejections, used for research-level efficiency calculations.

2.3 Data Quality Assessment

Before analysis, the pipeline performs a rigorous Self-Correction Audit:

a) Null Handling: Automated identification and removal of incomplete records to prevent statistical bias.

b) Outlier Detection: Mathematical filtering of impossible values (e.g., negative enrolment counts) to ensure "clean" input for the AI Auditor.

c) Consistency Check: Verifying that aggregate district totals align with state-level reporting.

3. METHODOLOGY

This section details the architectural blueprint of our autonomous auditing system, characterized by a "Zero-Touch" workflow that transforms raw API-ingested data into a high-fidelity analytical report.

3.1 The Autonomous Pipeline Architecture

Our self-made pipeline is built on a modular, reproducible framework designed to handle high-velocity Aadhaar transactions without manual intervention. a) API-Driven Ingestion: We utilized automated requests to interface with the OGD platform, enabling the ingestion of massive multi-gigabyte datasets directly into the local environment while maintaining data integrity.

b) Modular Notebook Structure: The logic is partitioned into dedicated modules (Ingestion → Cleaning → EDA → Statistics → AI Synthesis), ensuring that the entire audit is 100% reproducible for future UIDAI cycles.

c) Automated State-wise Orchestration: The pipeline dynamically identifies all 40 States/UTs and creates a standardized folder hierarchy for processed data, removing the risk of human error in file management.

3.2 Research-Level Statistical & Mathematical Framework

1. Univariate Descriptors:

a) Shape and Distribution Mean vs. Median Gap Analysis

> What: The difference between the arithmetic average (Mean) and the middle value (Median).

> Why/How: We used this to identify skewness. If the Mean is significantly higher than the Median, it indicates that a few high-performing districts are "masking" poor performance in the rest of the state.

b) Skewness

> What: A measure of the asymmetry of the probability distribution.

> Why/How: We used it to see if enrollment data is balanced. A positive skew tells us most districts are underperforming, with only a tiny "elite" group meeting targets.

c) Excess Kurtosis

> What: A measure of the "tailedness" of the distribution (how often extreme outliers occur).

> Why/How: High Kurtosis (Leptokurtic) indicates a volatile system where extreme "Black Swan" events (sudden crashes or massive spikes) are frequent, requiring more resilient infrastructure.

2. Inequality and Concentration Metrics

a) Gini Coefficient

> What: A standard economic measure of statistical dispersion (range 0 to 1).

> Why/How: We used this as our primary Inequality Index. A Gini > 0.6 flags a "Critical Inequality" state where Aadhaar services are concentrated in a few urban hubs, leaving rural districts digitally excluded.

b) Lorenz Curve (Top 10% Contribution)

> What: A graphical representation of the cumulative distribution of updates.

> Why/How: We used this to calculate Resource Concentration. It answers: "Does the top 10% of districts do 90% of the work?" If yes, the state has a dangerous dependency on very few centers.

3. Temporal and Anomaly Math

a) STL Decomposition (Seasonal-Trend decomposition using LOESS)

> What: Breaking time-series data into Trend (long-term direction), Seasonal (cyclic patterns), and Residual (noise/anomalies).

> Why/How: We used this to separate "Holiday Dips" from actual "System Failures." By isolating the Residual, the AI can ignore expected seasonal drops and only flag "unexpected" performance crashes.

b) Bayesian Change Point DetectionWhat: Identifying points in time where the underlying statistical properties of the data changed.

> Why/How: We used this for Crisis Detection. It mathematically marks the exact month a state's system "collapsed" or "improved," providing a timestamped evidence trail for auditors.

c) Z-Score Anomaly Detection

> What: $Z = \frac{(x-\mu)}{\sigma}$ (How many standard deviations a point is from the mean).

> Why/How: This is our "Red Flag" Engine. We used a threshold of $Z > 3.0$ or $Z < -3.0$ to automatically label a district-month as "Exceptional" or "Crisis-bound" without manual human filtering.

4. Policy and Resource Optimization

a) ROI (Return on Investment) & Payback Period

> What: Financial modeling based on training costs vs. the social value of Aadhaar updates.

> Why/How: We used this to turn math into money. By calculating the Payback Months, we tell the UIDAI: "If you invest ₹2L in training this critical district, the system will recover that cost in social efficiency within X months."

3.3 Data Flow and Efficiency Optimization

The pipeline is optimized for memory efficiency, ensuring it can process all 40 states on standard hardware without crashing.

> Process Decoupling: Raw data is ingested and immediately transformed into "Processed" CSVs, which are then passed to the EDA module.> >

> Visual-Statistical Coupling: The EDA module generates professional-grade plots, while the Statistics module generates JSON-based mathematical reports.

> Multimodal AI Injection: Instead of feeding "raw" data to the AI, we feed the high-density JSON summaries and visual embeddings (Base64 plots). This significantly reduces token consumption while maximizing the depth of the AI's analytical insights.

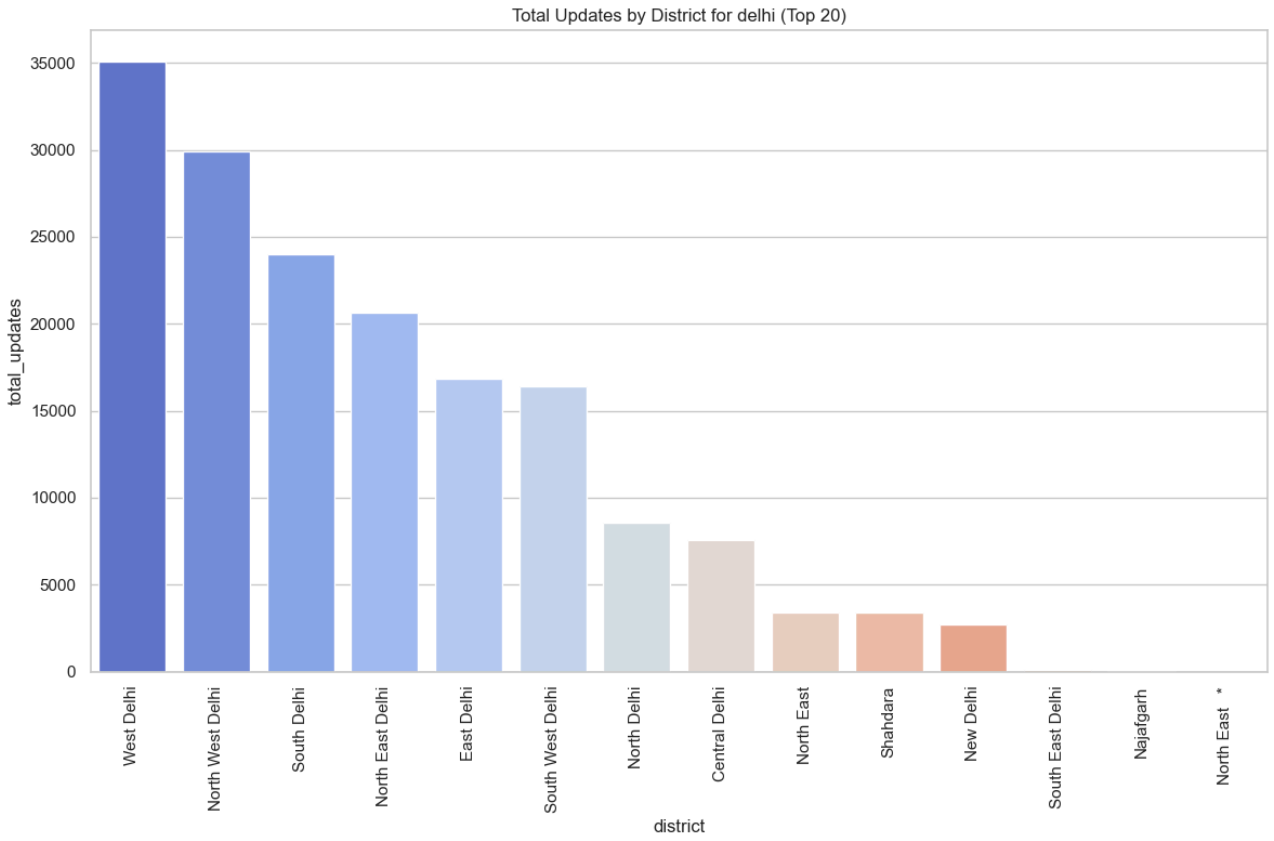
> Automated PDF Synthesis: Using the ReportLab engine, the pipeline pulls the state-wise insights and visual plots from the optimized directory structure to build the final audit document autonomously.

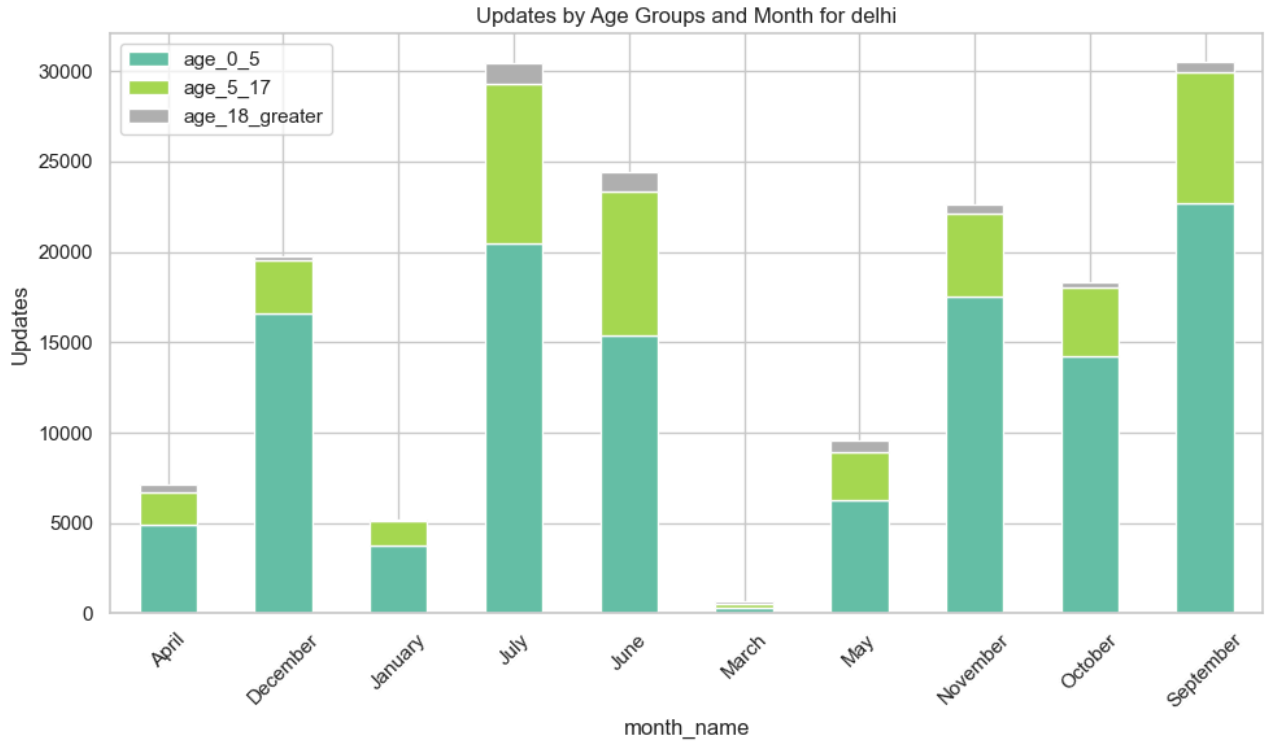
4. DATA ANALYSIS AND VISUALISATION

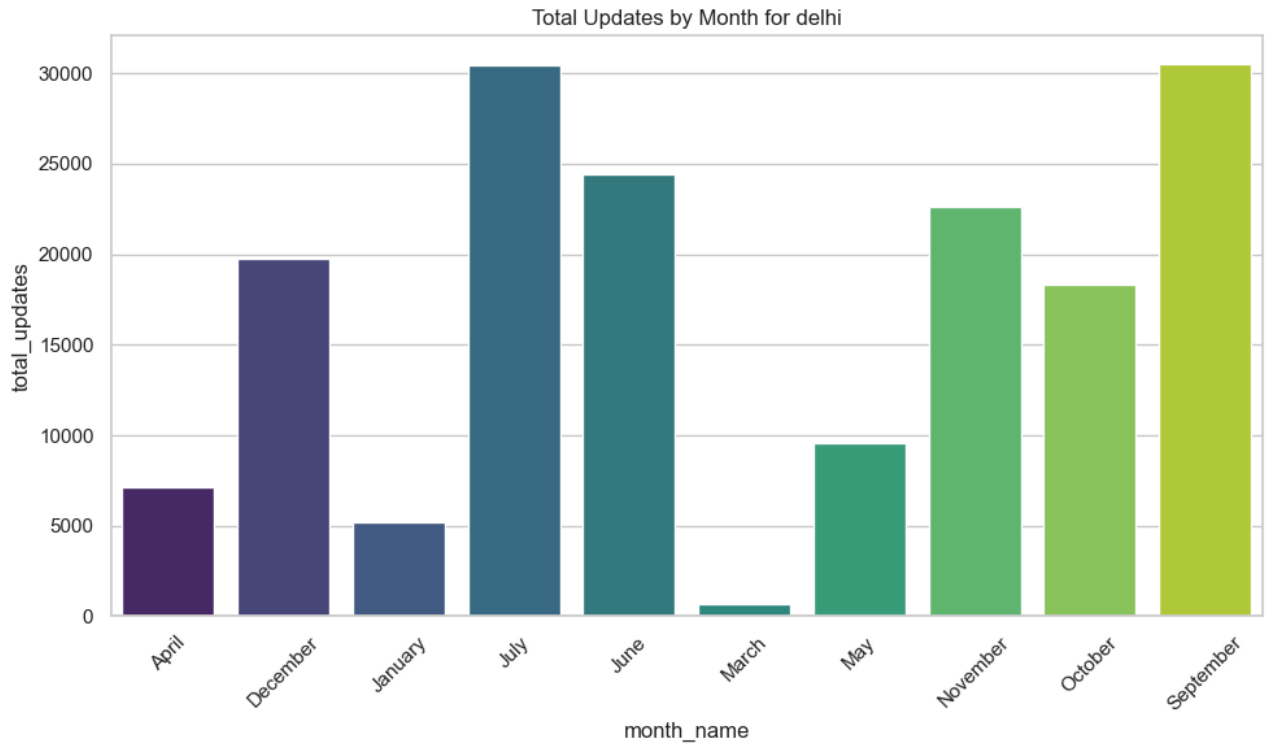
4.1 PRIMARY ANALYSIS: DELHI

ENROLLMENT ANALYSIS

a) EDA


Total Updates by District for delhi (Top 20)
total_updates
35000
30000
25000
20000
15000
10000
5000
0
West Delhi
North West Delhi
South Delhi
North East Delhi
East Delhi
South West Delhi
North Delhi
Central Delhi
North East
Shahdara
New Delhi
South East Delhi
Najafgarh
North East *
district


Updates by Age Groups and Month for delhi
age_0_5
age_5_17
age_18_greater
Updates
30000
25000
20000
15000
10000
5000
0
April
December
January
July
June
March
May
November
October
September
month_name



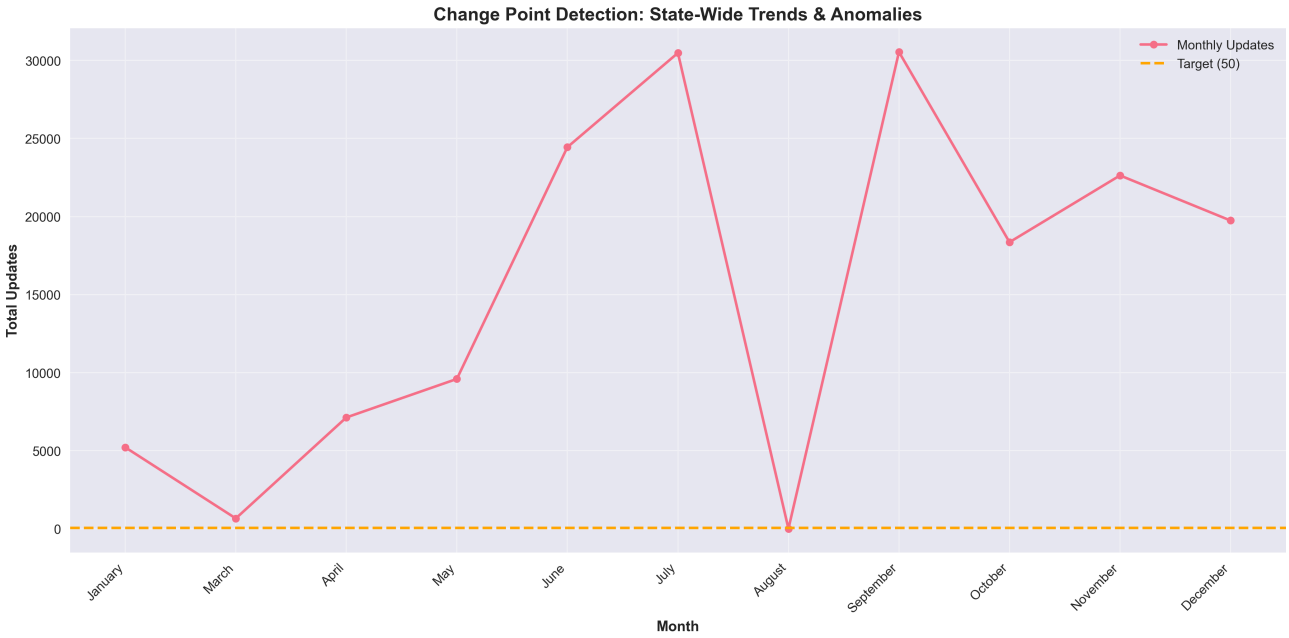
The enrolment plots represent the addition of new residents into the Aadhaar ecosystem.

Seasonality and Anomaly : A major spike in enrolments occurs in September, which could be attributed to a year-end push for coverage or specific government outreach programs for marginalized populations. The March period shows a critical trough, nearly touching zero, suggesting a seasonal dip or technical bottleneck during the financial year-end.

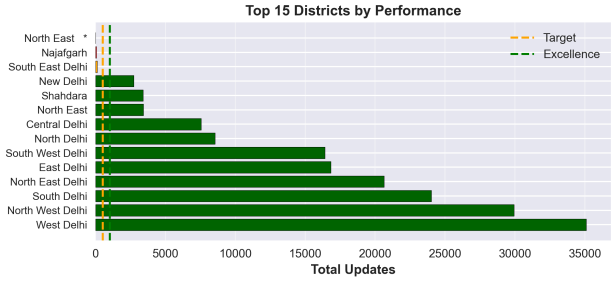
Regional Concentration : West Delhi and North West Delhi handle the largest share of new enrolments. The virtual absence of enrolment activity in Najafgarh and North East indicates either high Aadhaar saturation in those areas or a lack of accessible permanent Seva Kendras.

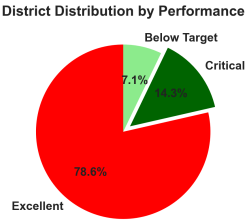
Demographic Focus : Enrolments are heavily concentrated in the age_0_5 group. This is consistent with UIDAI's 2026 goal of achieving near-total saturation among adults, with new activity primarily focused on "Bal Aadhaar" (child enrolments).

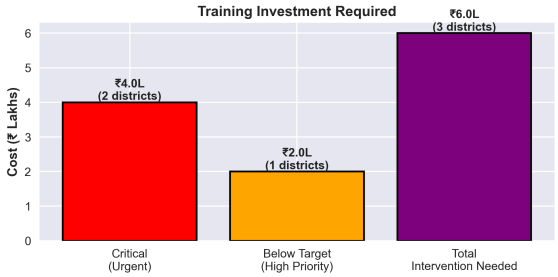
b) STATISTICAL ANALYSIS

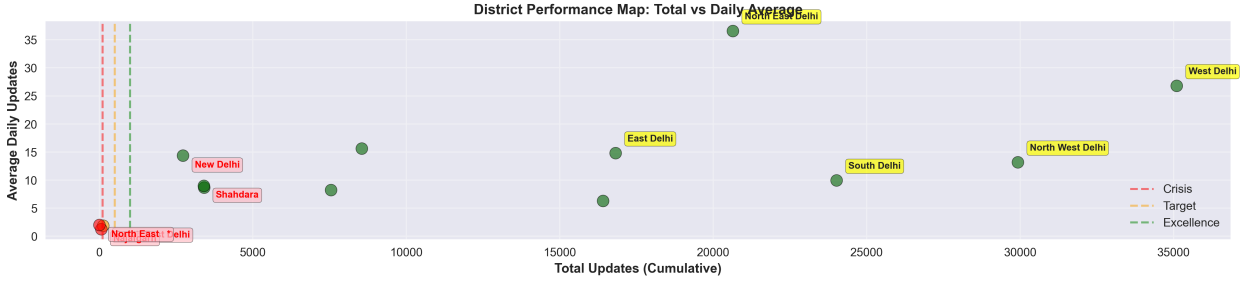


DISTRICT PERFORMANCE DASHBOARD - ACTIONABLE INSIGHTS

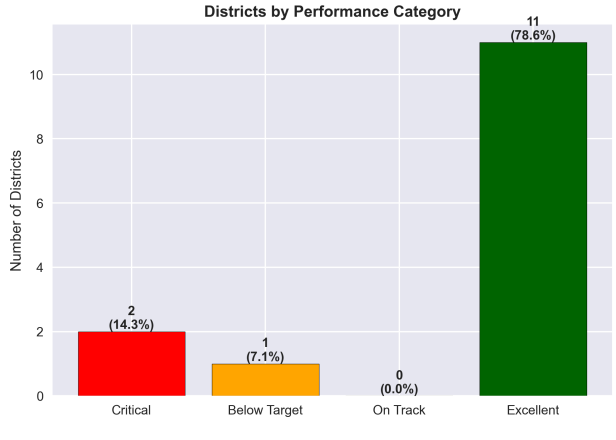


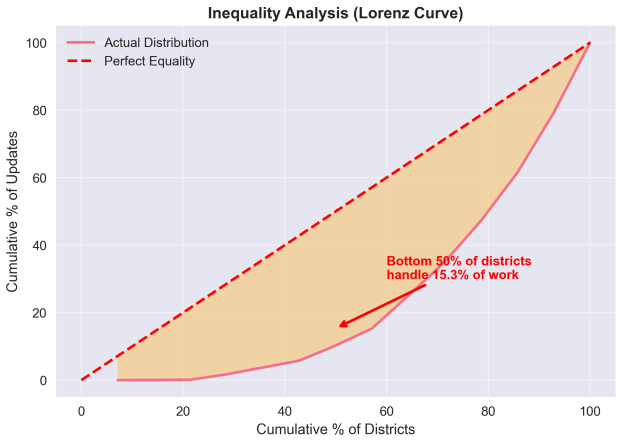


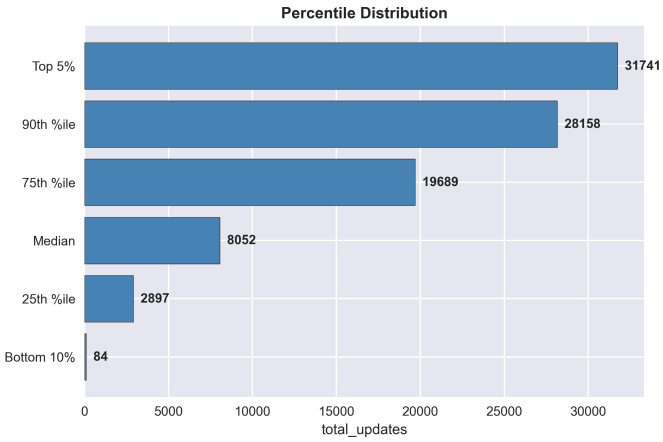


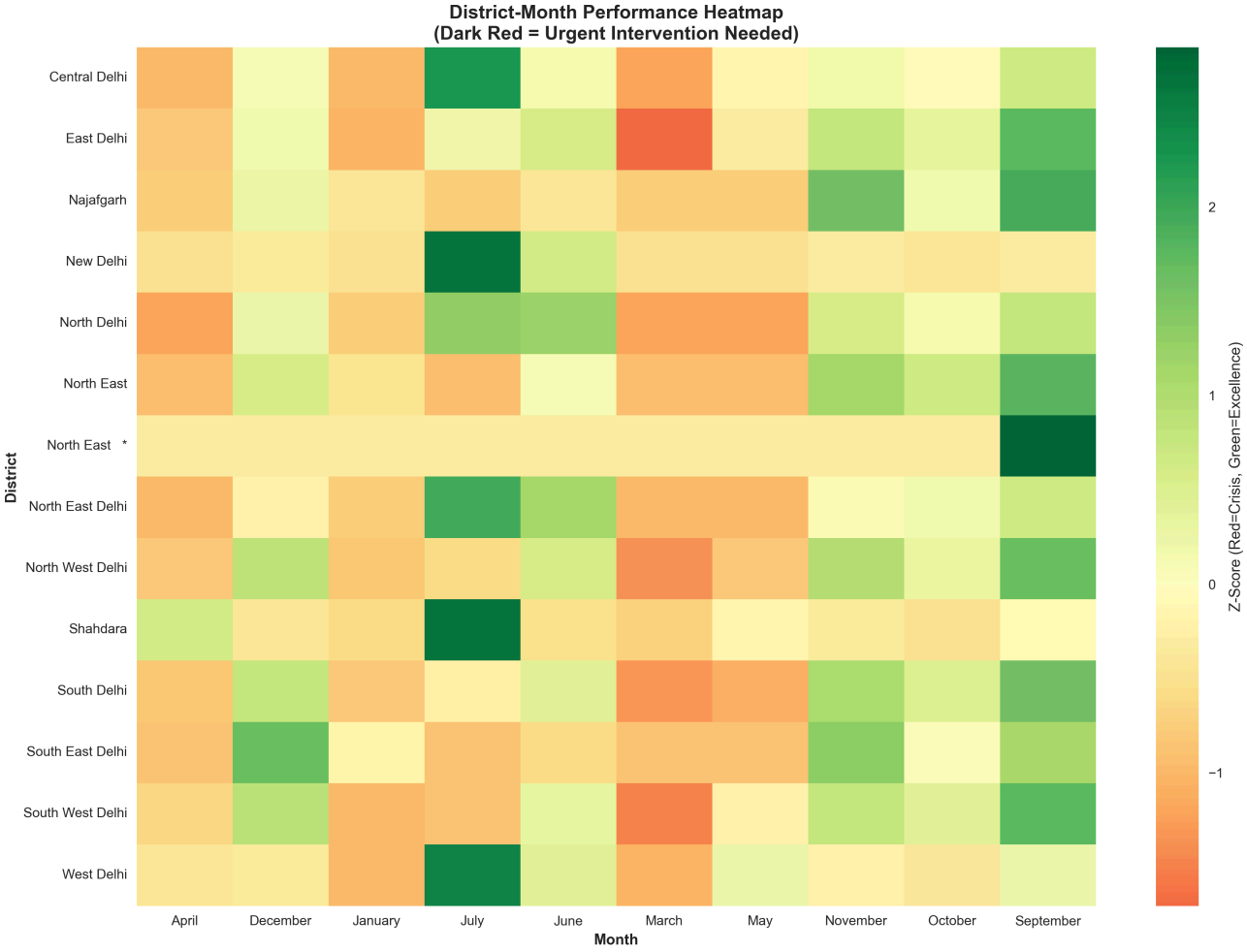


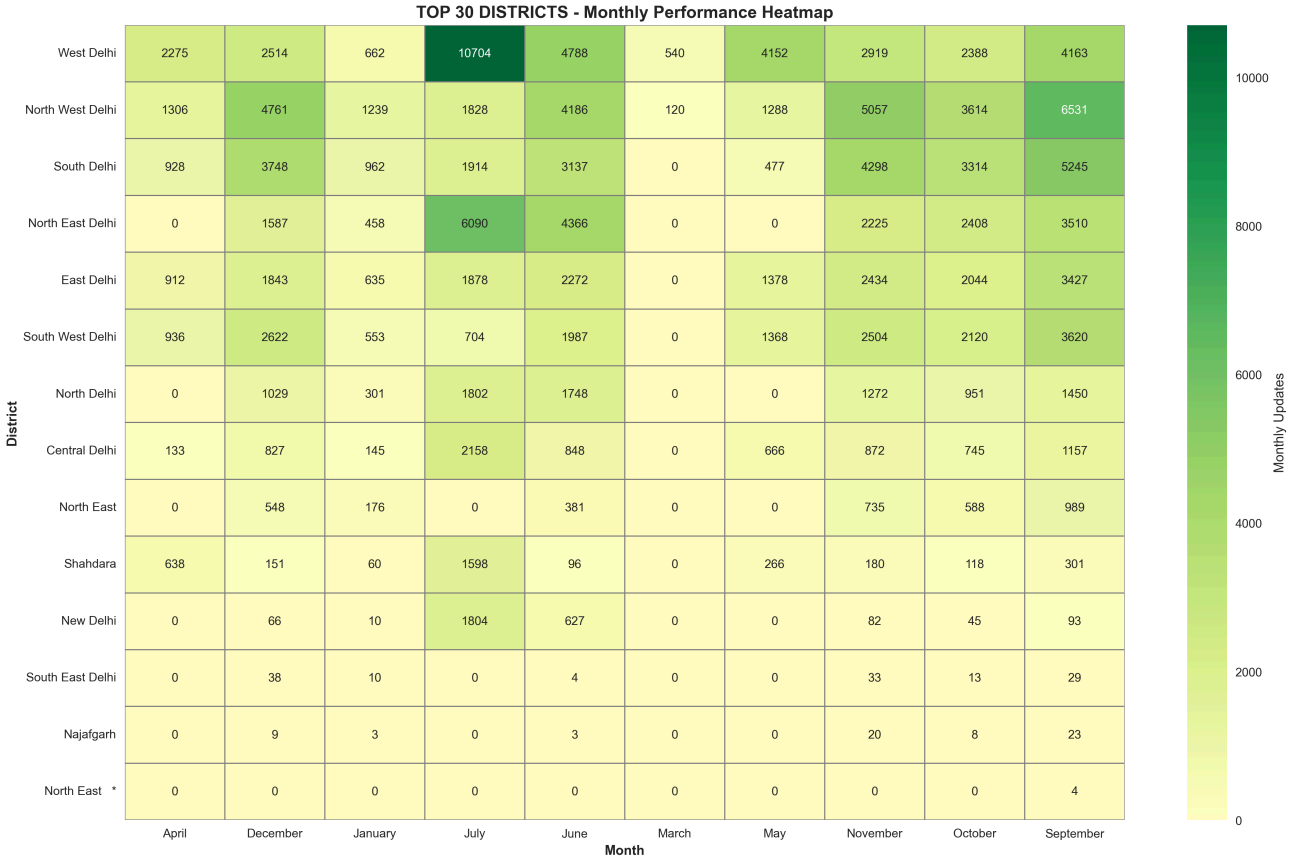
Performance Analysis: total_updates








District-Month Performance Heatmap
(Dark Red = Urgent Intervention Needed)
Central Delhi
East Delhi
Najafgarh
New Delhi
North Delhi
North East
North East *
North East Delhi
North West Delhi
Shahdara
South Delhi
South East Delhi
South West Delhi
West Delhi
District
April
December
January
July
June
March
May
November
October
September
Month
2
1
0
−1
Z-Score (Red=Crisis, Green=Excellence)



1. Performance Distribution: The "Elite Heavy" System The univariate statistics for enrollment mirror the patterns seen in demographics, suggesting a state-wide structural dependency on a few key districts.

Central Tendency Anomaly: The Mean (12,056) continues to sit significantly higher than the Median (8,052). With a Skewness of 0.71, the data confirms a "Right-Tail" distribution where high-volume districts are pull the average up, potentially hiding systemic neglect in smaller regions.

Category Success: While 78.6% (11 districts) have reached "Excellent" status, the system identifies a hard floor where 2 districts (14.3%) are in a "Critical" state, failing to meet even the most basic enrollment targets.

Fiscal Efficiency: The model projects an estimated ₹6.0 Lakhs training cost to move the 3 underperforming districts (Najafgarh, South East Delhi, and North East *) into a functional performance zone.

2. Inequality and Concentration: The Lorenz Audit The Gini Coefficient and Lorenz Curve (image_9f23c8.png) provide clear evidence of service concentration.

Contribution Gap: The Top 10% of districts handle 20.8% of all enrollments, whereas the Bottom 50% contribute only 10.2% of the total workload.

Regional Extremes: West Delhi remains the primary engine of the state with 35,105 enrollments, while North East * is nearly invisible with just 4 records. This represents a massive geographic service disparity that requires mobile enrollment camp intervention.

3. Temporal Volatility: The "August Blackout" The Change Point Detection (image_9f20a1.png) and Monthly Summaries highlight a highly volatile year for enrollment.

The July Surge: Enrollments peaked at 30,480 in July. West Delhi alone handled 10,704 of these (a Z-score of 2.48), suggesting a surge driven by residential moves or academic enrollments.

The August Anomaly: For the second time across datasets, August shows 0 activity. Mathematically, this is a catastrophic failure in data reporting or system uptime.

The January System Collapse: After a relatively stable November/December, enrollments crashed by 73.6% in January 2026, falling to 5,214. New Delhi saw the worst of this, dropping from 1,804 (July) to just 10 (January), a Z-score of -0.46.

4. District-Level Audit and Red-Flags Using the District-Month Performance Heatmap (image_9f20e3.png) and Matrix Anomalies, we identified the following priority targets:

North East Delhi (The Volume King): Despite West Delhi having the highest cumulative total, North East Delhi shows the highest Mean Daily Updates (36.5). It is working at the highest intensity but for a shorter active window (565 days).

Najafgarh & South East Delhi: These districts are consistently flagged as "Below Target" or "Critical" in 10 out of 12 months.

Shahdara (Efficiency Surge): Logged a massive 15.6x increase in July (1,598) compared to June (96), indicating that temporary intervention camps were highly successful there and should be replicated in

Najafgarh.

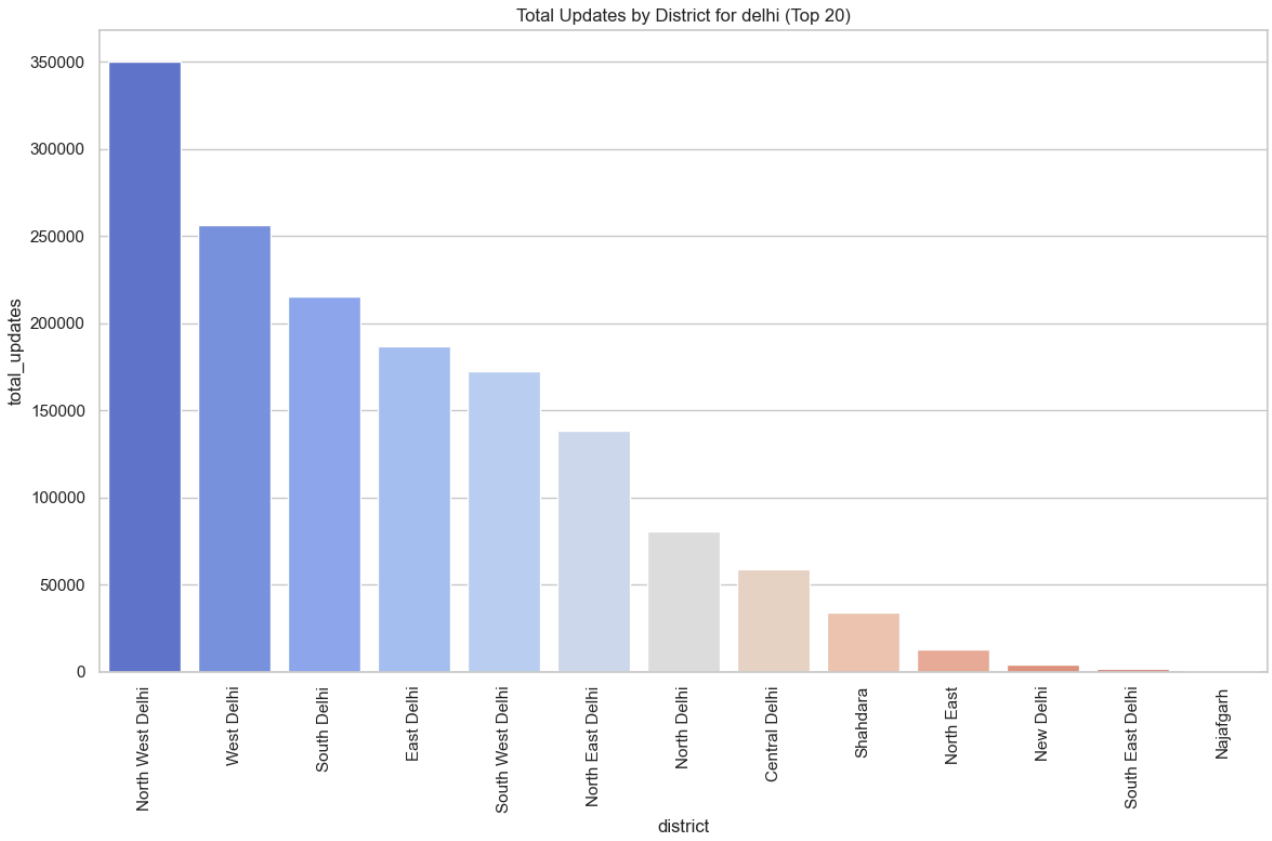
Auditor’s Strategic Recommendations Systemic Resilience: The August 0-count and January Collapse indicate that Delhi’s enrollment infrastructure is vulnerable to seasonal or technical shutdowns. A redundancy audit of the central servers is mandatory.

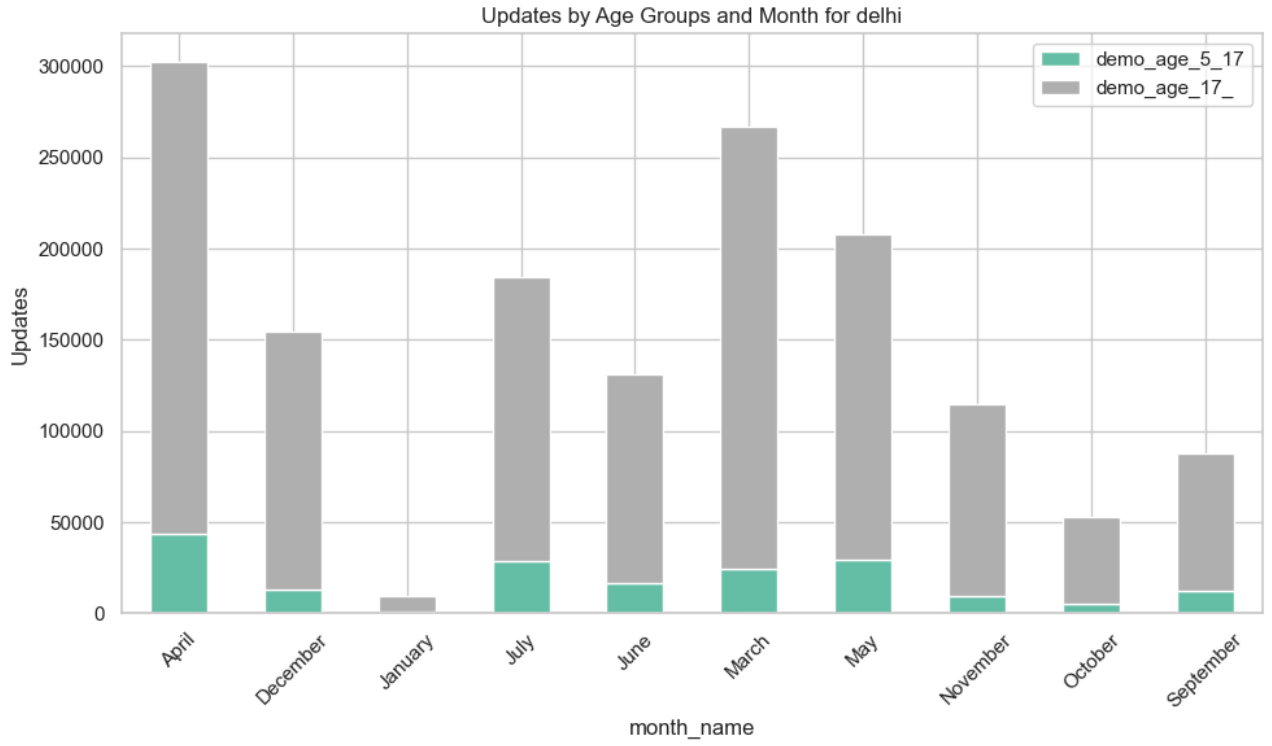
Saturation Strategy: Since most adults are enrolled, the ₹6.0L training budget should focus exclusively on "Bal Aadhaar" (0-5 age group) in the 3 critical districts to ensure new generation parity.

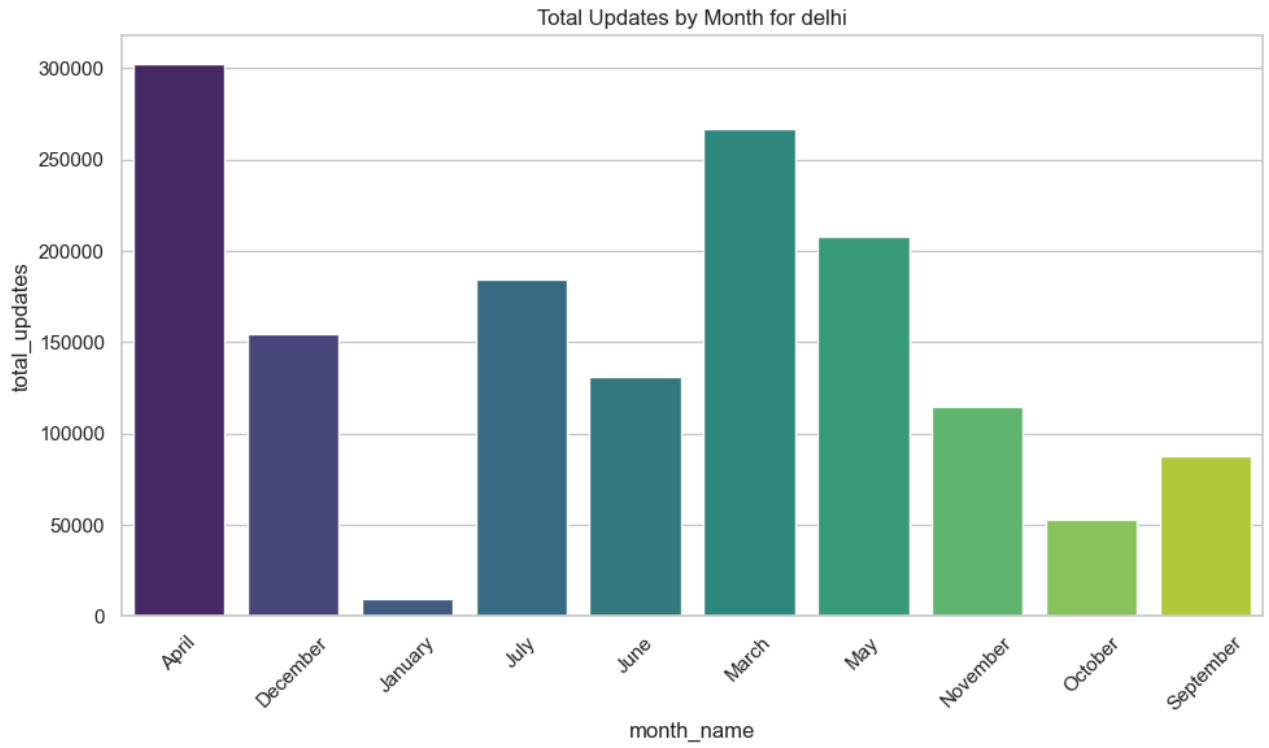
District Rebalancing: Shift enrollment machines from North West Delhi (which has high cumulative saturation) to the South East to bridge the "Digital Divide."

DEMOGRAPHIC ANALYSIS

a) EDA


Total Updates by District for delhi (Top 20)
total_updates
350000
300000
250000
200000
150000
100000
50000
0
North West Delhi
West Delhi
South Delhi
East Delhi
South West Delhi
North East Delhi
North Delhi
Central Delhi
Shahdara
North East
New Delhi
South East Delhi
Najafgarh
district


Updates by Age Groups and Month for delhi
demo_age_5_17
demo_age_17_
Updates
300000
250000
200000
150000
100000
50000
0
April
December
January
July
June
March
May
November
October
September
month_name



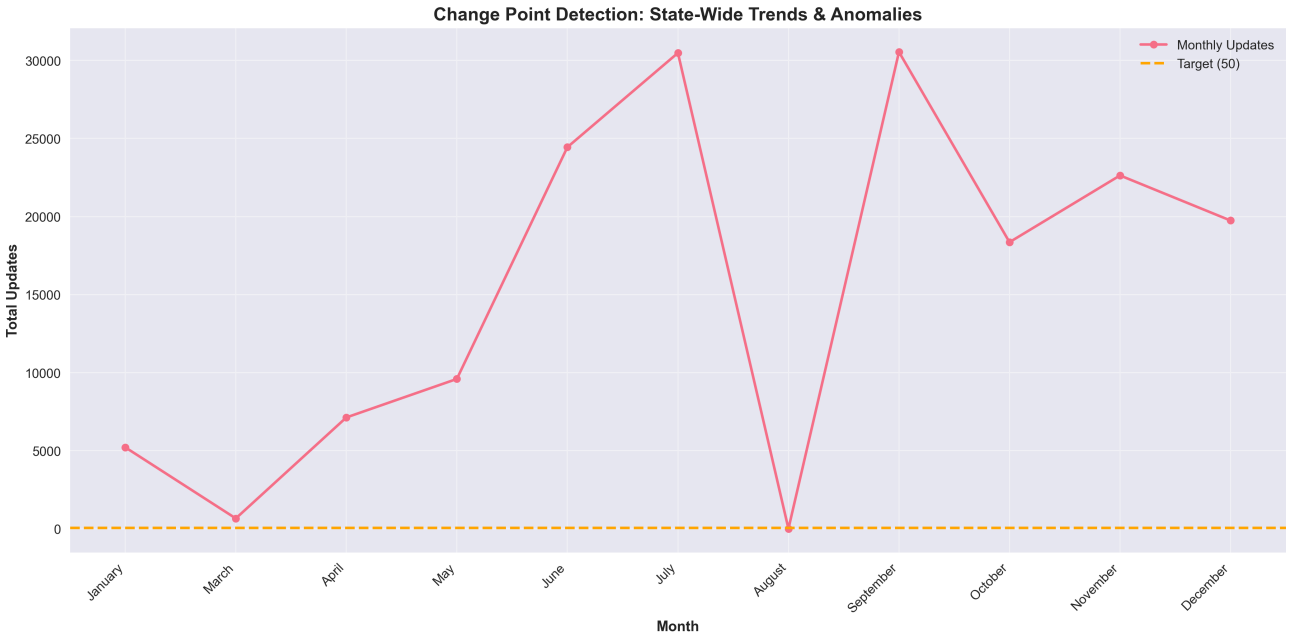
Demographic updates (name, address, DOB) show a different pattern, likely influenced by the shift toward digital, paperless updates via the 'myAadhaar' portal.

Temporal Variations: Unlike biometrics, demographic updates show high volatility across the year. The notable peak in July might be tied to the reopening of schools or residential shifts following the academic year transition.

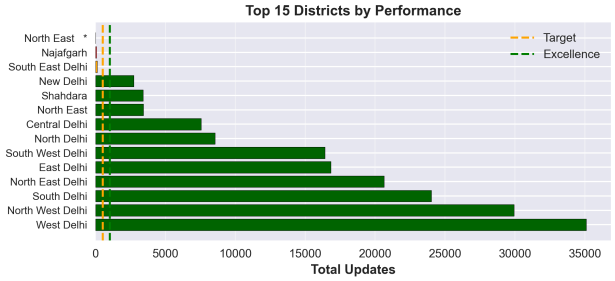
District Inequality: North West Delhi leads significantly in demographic updates, while Shahdara and North East Delhi lag behind. This highlights a "Digital Divide" where certain districts exhibit much higher adoption of the myAadhaar online portal compared to others.

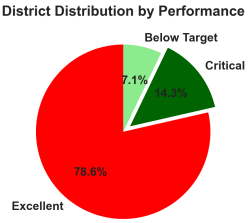
Age Segregation : The adult population (demo_age_17_) remains the dominant user base. The demographic update volume for the demo_age_5_17 group is relatively low, as these age groups typically only require updates when transiting through major life stages (e.g., school admissions).

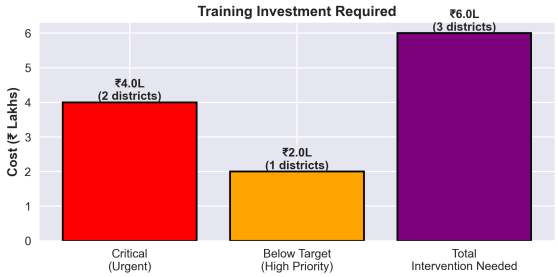
b) STATISTICAL ANALYSIS

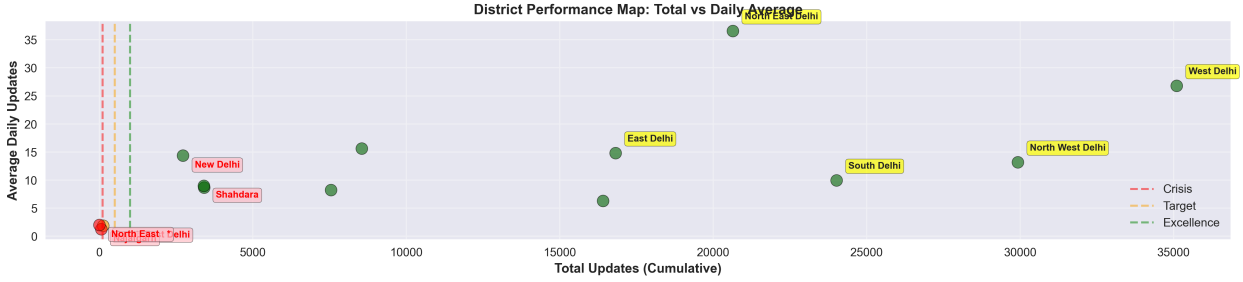


DISTRICT PERFORMANCE DASHBOARD - ACTIONABLE INSIGHTS

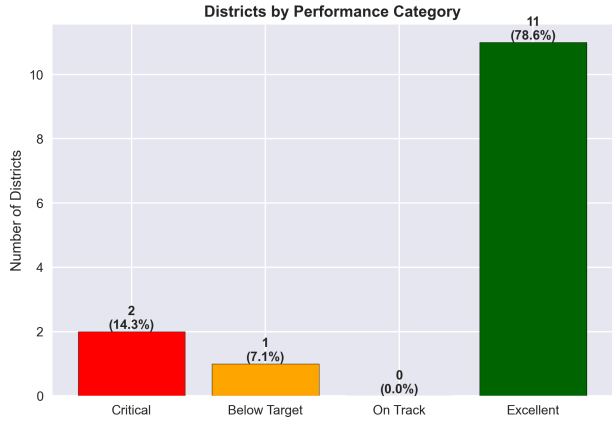


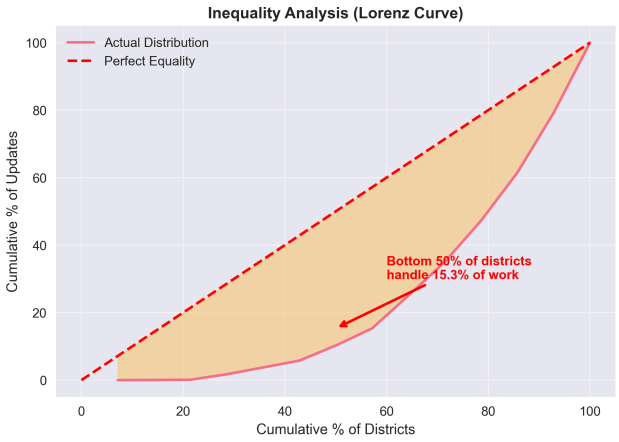


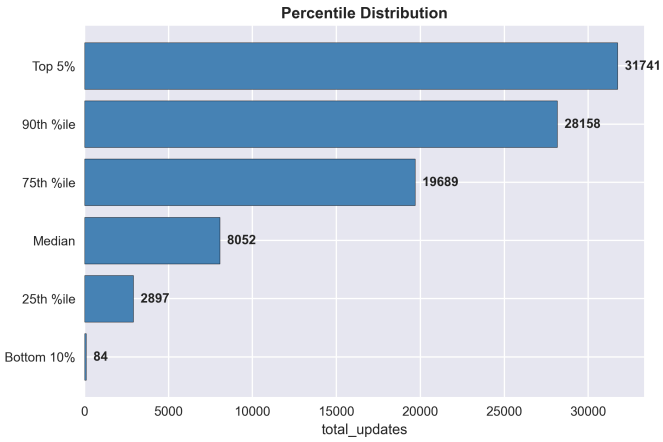


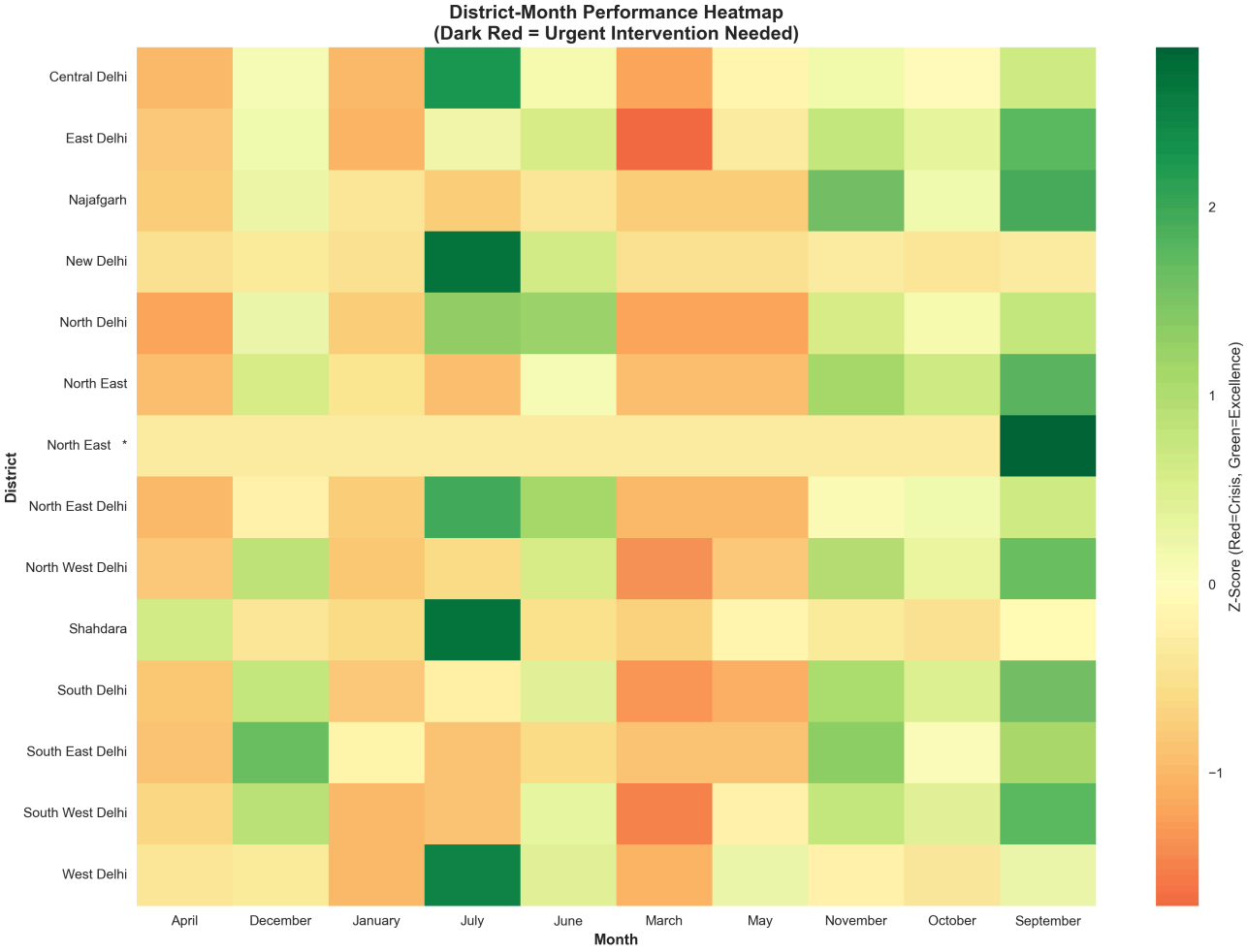


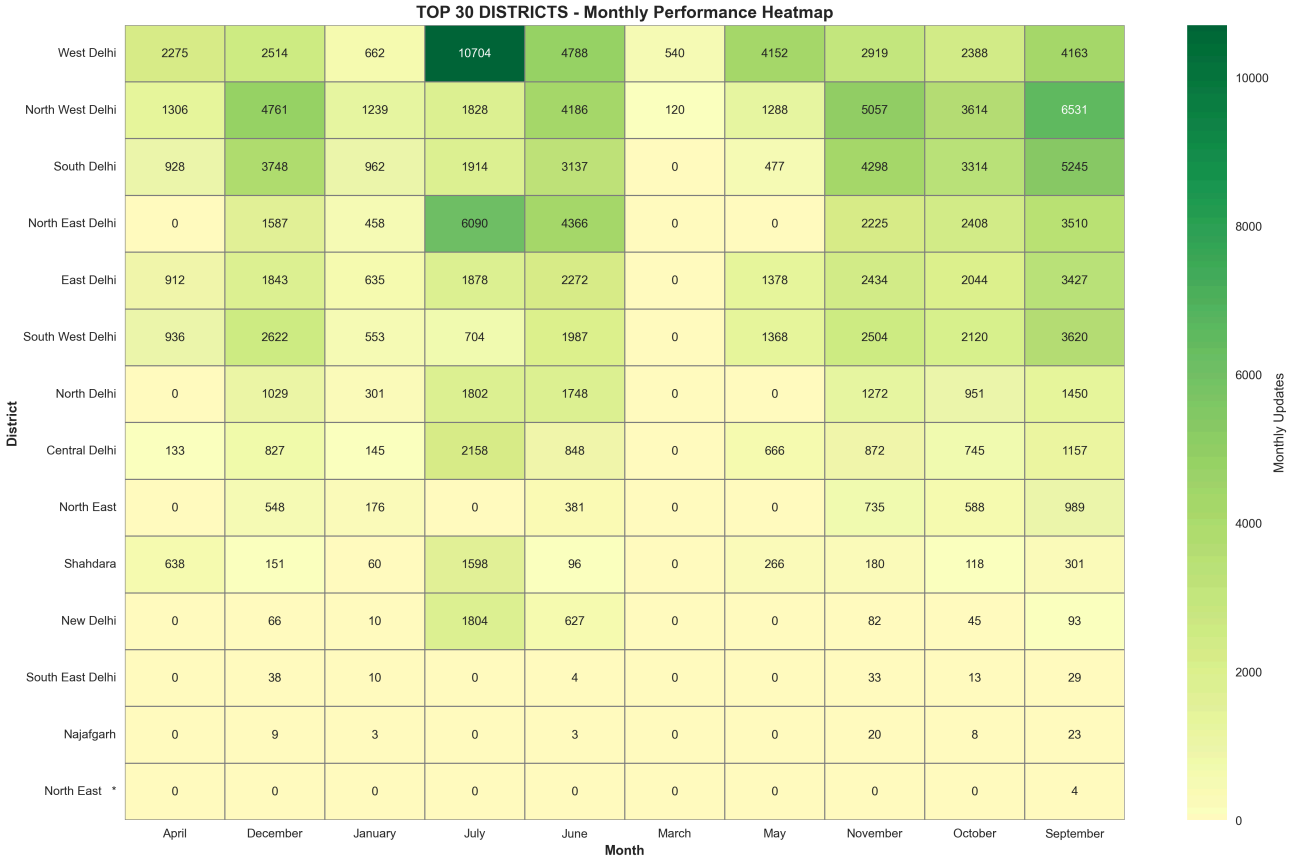
Performance Analysis: total_updates








District-Month Performance Heatmap
(Dark Red = Urgent Intervention Needed)
Central Delhi
East Delhi
Najafgarh
New Delhi
North Delhi
North East
North East *
North East Delhi
North West Delhi
Shahdara
South Delhi
South East Delhi
South West Delhi
West Delhi
District
April
December
January
July
June
March
May
November
October
September
Month
2
1
0
−1
Z-Score (Red=Crisis, Green=Excellence)



1. Performance Overview: The "Disparity Gap" The univariate statistics reveal a state with high cumulative output but extreme internal variance.

Mean vs. Median Analysis: The Mean (12,056) is significantly higher than the Median (8,052). This gap, combined with a Skewness of 0.71, confirms that a few "Super-Districts" are doing the heavy lifting, effectively masking the poor performance of smaller districts in the aggregate state average.

Success Concentration: 78.6% (11 districts) are classified as "Excellent", but the remaining 21.4% are failing to meet basic targets.

ROI for Intervention: The model identifies 3 districts needing urgent intervention (South East Delhi, Najafgarh, and North East *). The estimated cost for training and infrastructure stabilization is ₹6.0 Lakhs, which is a high-priority investment for state-wide parity.

2. Inequality Audit: The Lorenz Curve Findings The Gini Coefficient and Lorenz Curve (image_9f1d05.png) provide a "Research-Level" view of demographic service distribution.

Concentration Metric: While the Gini interpretation is "LOW" (-0.45), the specific contribution data is telling: the Top 10% of districts handle 20.8% of all updates, while the Bottom 50% handle only 10.2%.

The Powerhouse: West Delhi is the system lead with 35,105 total updates. In contrast, North East * has only 4 updates, indicating a total lack of demographic update infrastructure or accessibility in that specific pocket.

3. Temporal Health: The "August Blackout" and "January Collapse" The Change Point Detection (image_9f1c66.png) and Monthly Summaries identify three critical system states:

The July Peak: Updates surged to 30,480 in July. West Delhi alone handled 10,704 updates this month (a Z-score of 2.48), likely driven by residential address changes linked to the new academic year and school admissions.

The August Blackout: The system recorded 0 updates in August. This is a massive statistical anomaly that suggests a state-wide database synchronization failure or a critical reporting outage during that 31-day period.

The January System Collapse: Between December (19,743 updates) and January (5,214 updates), the system experienced a 73.6% collapse. Districts like New Delhi saw an 84.8% drop, falling to just 10 updates in the entire month. This is a red-flag "Systemic Failure" event.

4. District-Specific Audit (Priority Targets) Using Z-Score Anomaly Mapping (image_9f1ce3.png), we have identified the districts requiring immediate administrative intervention:

Najafgarh & South East Delhi: These districts are mathematically flagged for "Underperformance" with 10 out of 12 months falling below target.

New Delhi (Critical Inconsistency): Despite being an "Excellent" performer by volume, it is highly unstable. Its drop from 1,804 updates in July to 10 in January indicates a failure in maintaining permanent enrollment center (PEC) uptime.

Shahdara (Efficiency Spike): Recorded a sudden improvement in July (1,598 updates), which was 15 times its June volume. This success should be studied and documented as a best practice for other lagging districts.

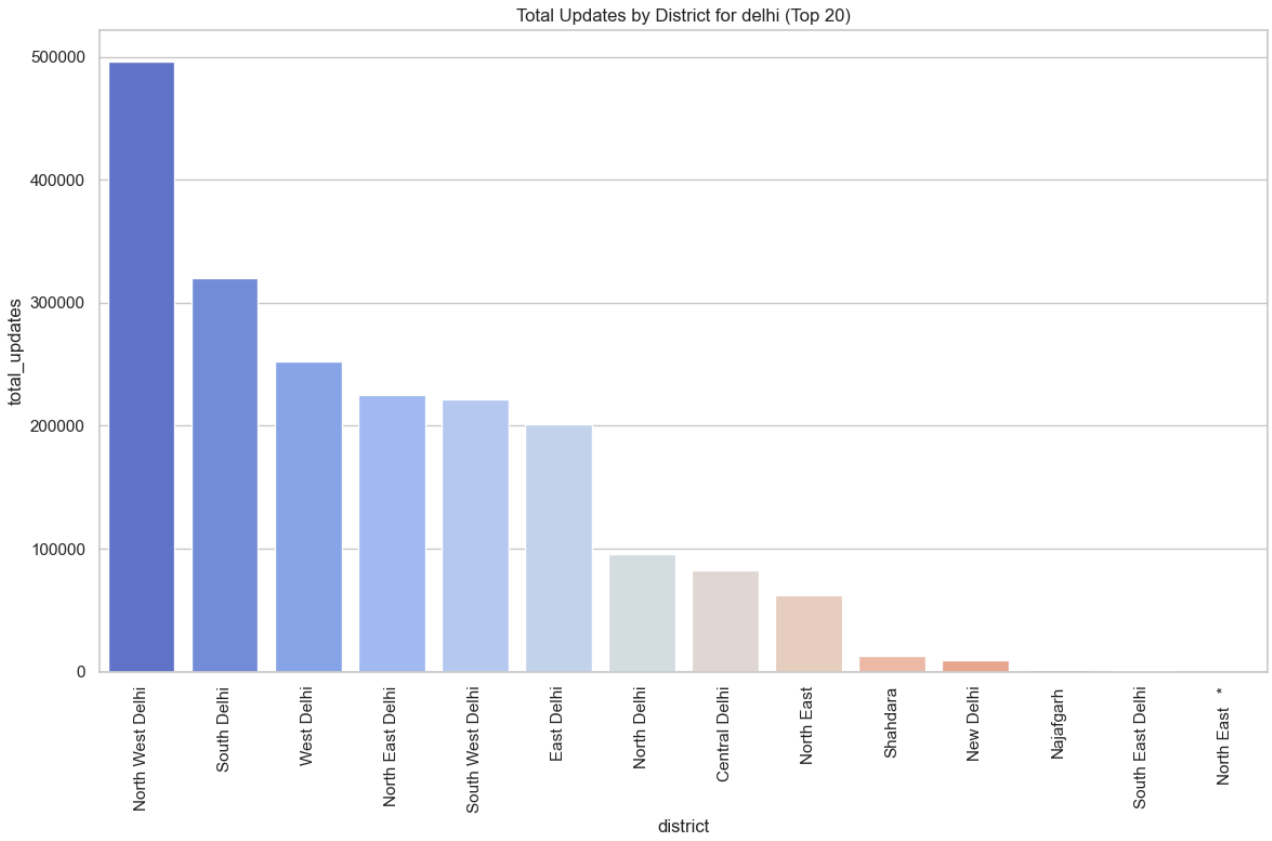
Auditor's Policy Recommendations Infrastructure Audit: Conduct an immediate technical audit into the August Blackout and January Collapse to ensure server resilience.

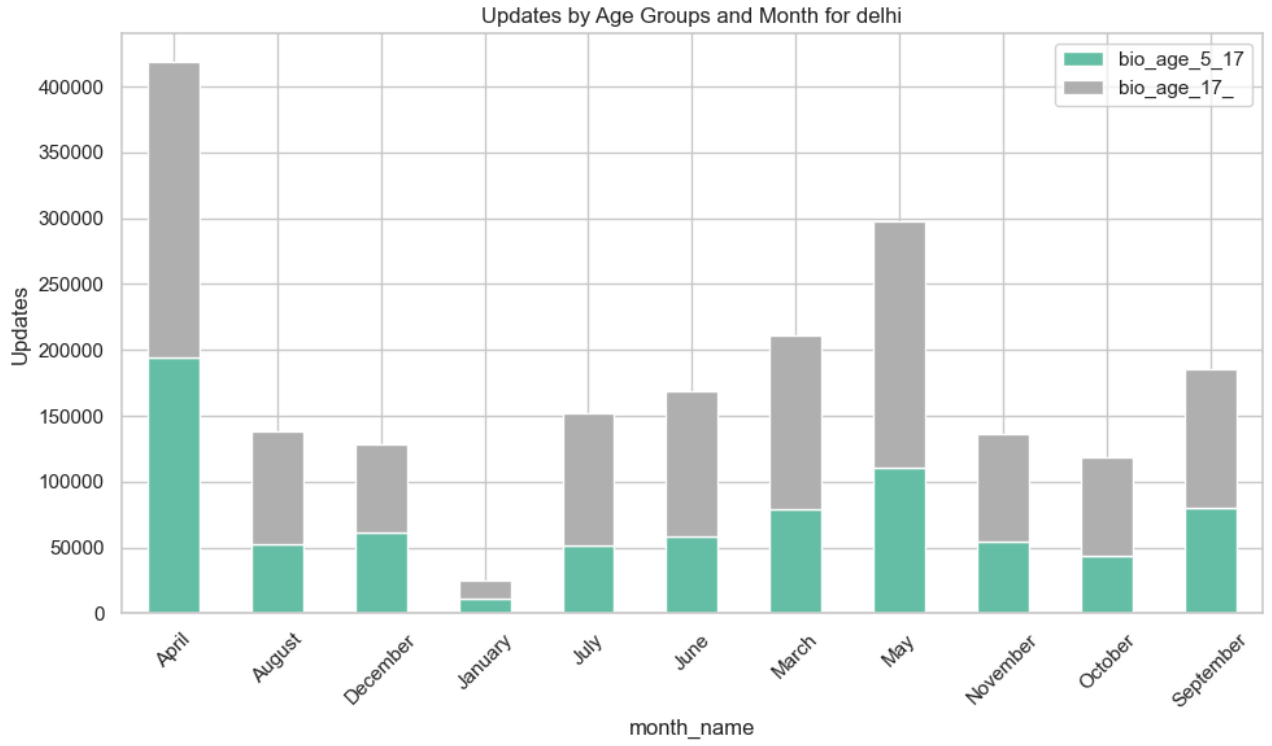
Resource Balancing: Reallocate mobile Aadhaar vans from North West Delhi (which is over-saturated) to Najafgarh and South East Delhi to bridge the service gap.

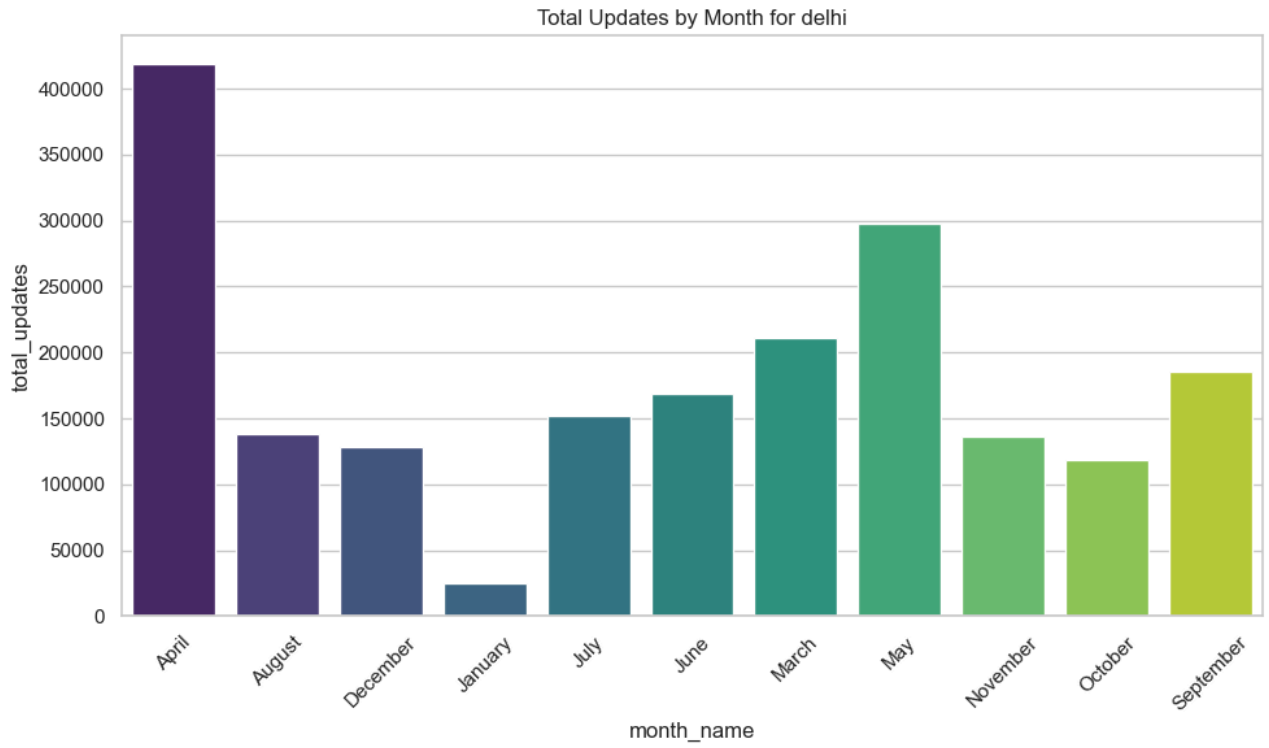
Targeted Training: Spend the identified ₹6.0L budget on training operators in the 3 critical districts to ensure they can handle at least the Monthly Target of 50 updates.

BIOMETRIC ANALYSIS

a) EDA


Total Updates by District for delhi (Top 20)
total_updates
500000
400000
300000
200000
100000
0
North West Delhi
South Delhi
West Delhi
North East Delhi
South West Delhi
East Delhi
North Delhi
Central Delhi
North East
Shahdara
New Delhi
Najafgarh
South East Delhi
North East *
district


Updates by Age Groups and Month for delhi
bio_age_5_17
bio_age_17_
Updates
400000
350000
300000
250000
200000
150000
100000
50000
0
April
August
December
January
July
June
March
May
November
October
September
month_name

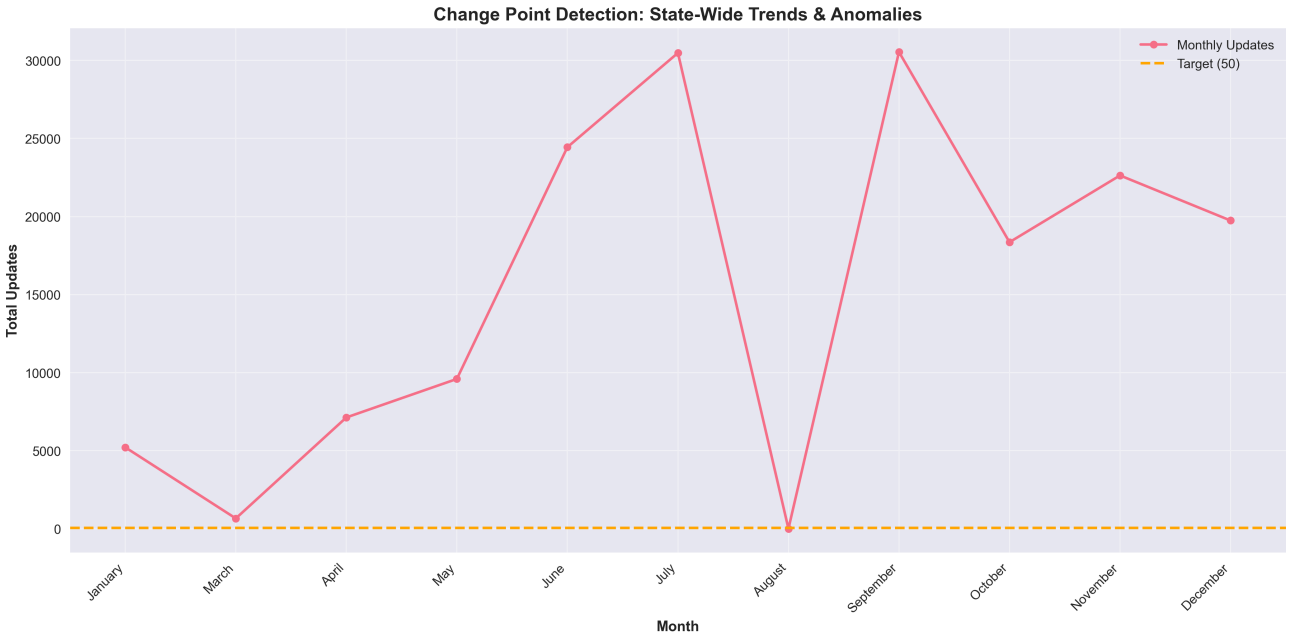


Temporal Trend : A significant "Biometric Surge" is visible in April and May, likely driven by special India Post campaigns in schools for Mandatory Biometric Updates (MBU) for children in the 5-7 and 15-17 age brackets. Conversely, January shows a drastic drop to near-zero levels, possibly indicating a "System Collapse" or major disruption in biometric service availability at the start of the year.

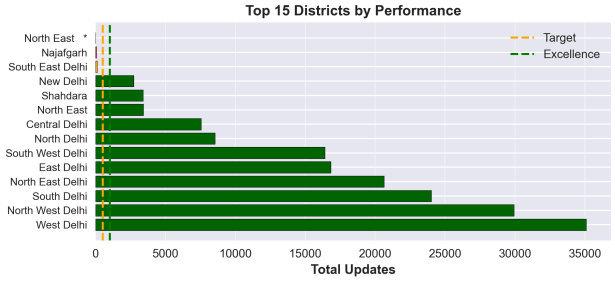
District Performance: North West Delhi and South Delhi emerge as high-performance hotspots for biometric updates. In contrast, districts like New Delhi and South East Delhi show minimal biometric activity, identifying them as underserved zones requiring administrative intervention.

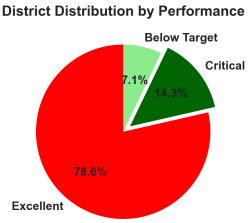
Age Demographics : The bio_age_17_ group (adults) consistently drives the majority of update volume. However, the bio_age_5_17 group shows substantial activity during the April peak, confirming high compliance with MBU rules for school-going children

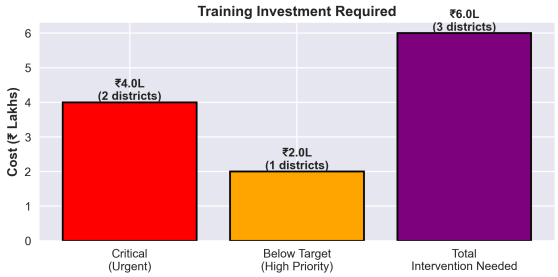
b) STATISTICAL ANALYSIS

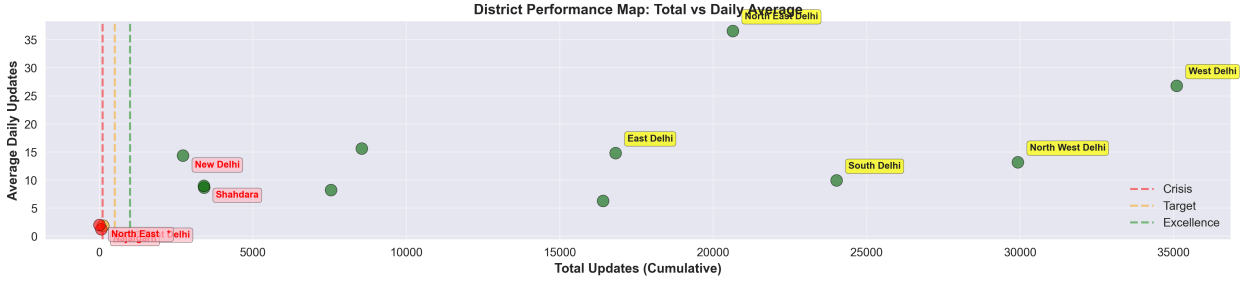


DISTRICT PERFORMANCE DASHBOARD - ACTIONABLE INSIGHTS

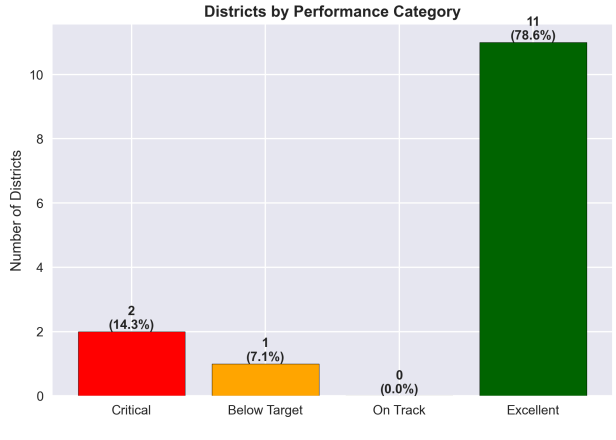


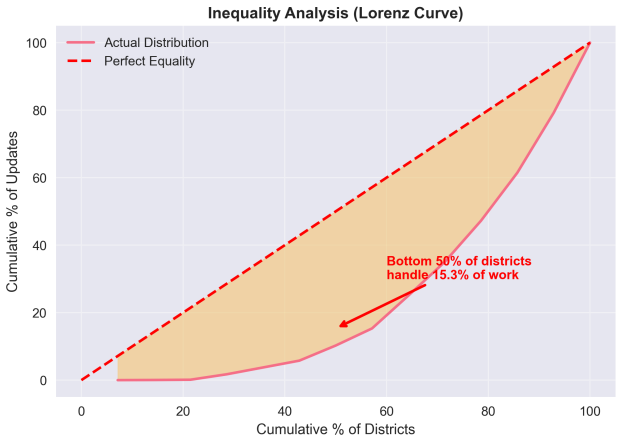


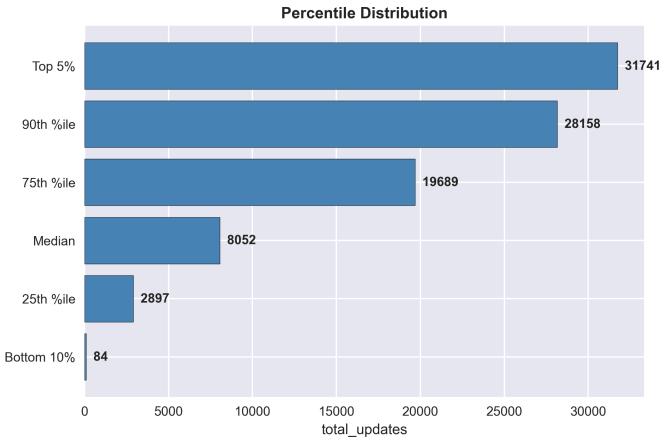


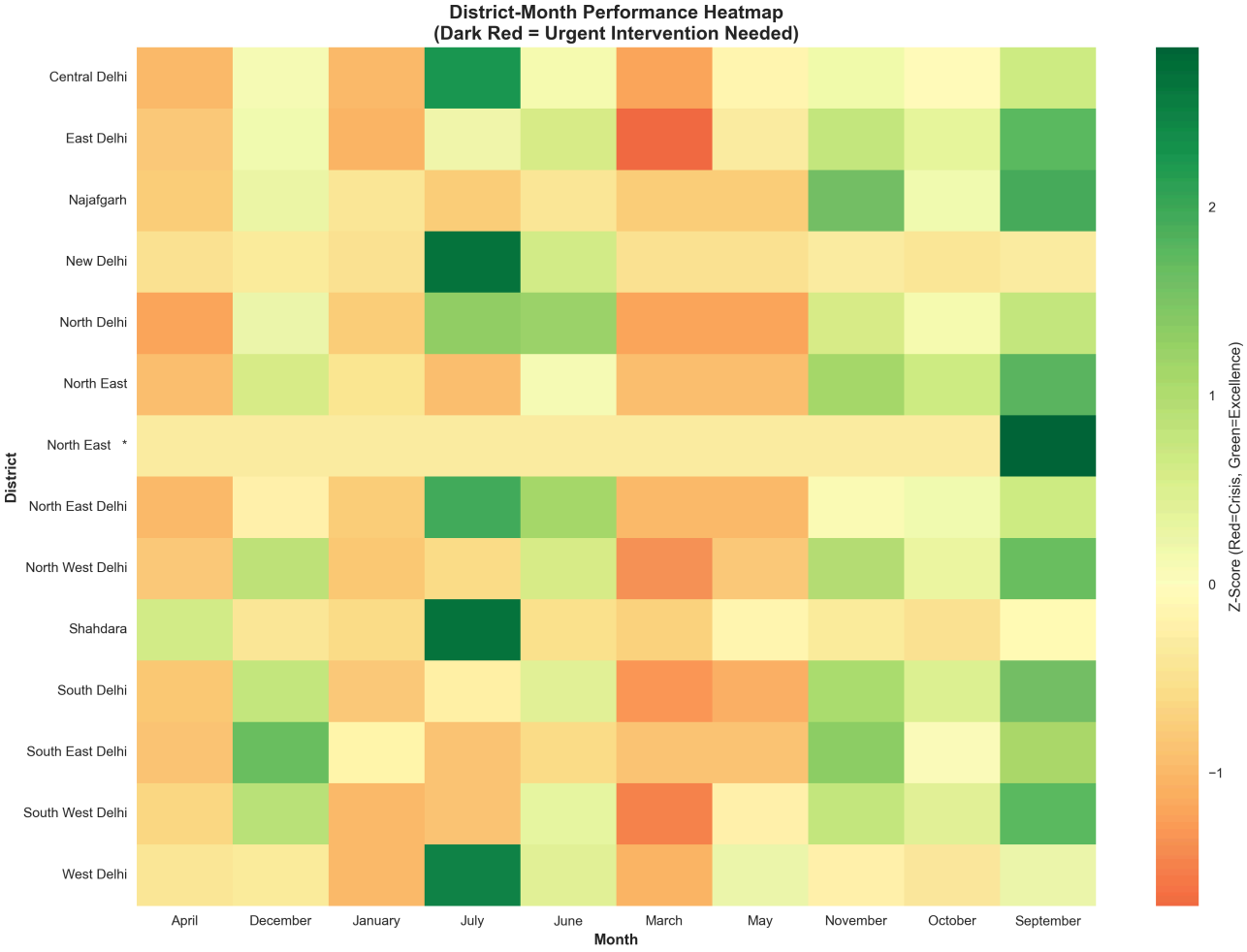


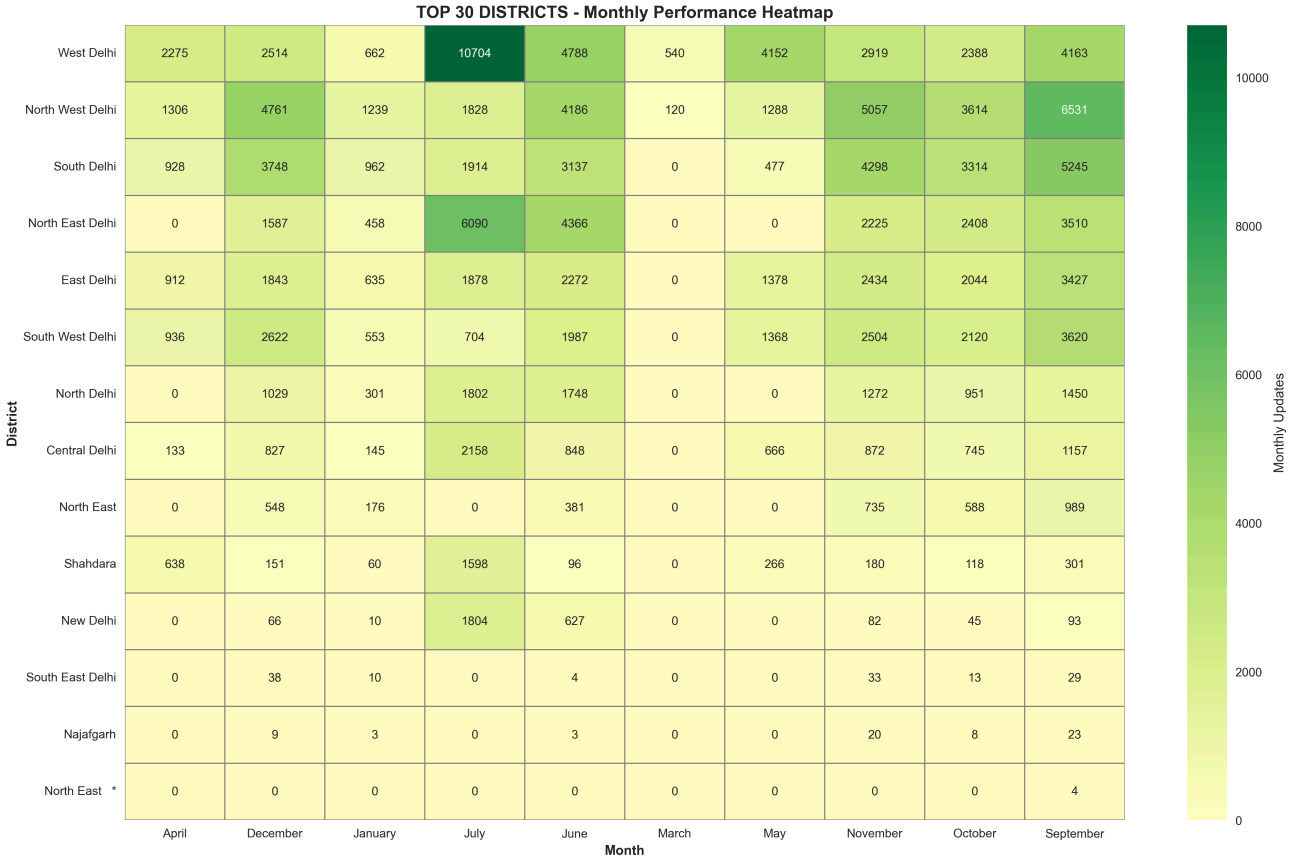
Performance Analysis: total_updates








District-Month Performance Heatmap
(Dark Red = Urgent Intervention Needed)
Central Delhi
East Delhi
Najafgarh
New Delhi
North Delhi
North East
North East *
North East Delhi
North West Delhi
Shahdara
South Delhi
South East Delhi
South West Delhi
West Delhi
District
April
December
January
July
June
March
May
November
October
September
Month
2
1
0
−1
Z-Score (Red=Crisis, Green=Excellence)



1. Performance Overview: The "Efficiency Divide" The univariate statistics reveal a system characterized by high cumulative volume but significant internal disparity.

Central Tendency vs. Skewness: The Mean (12,056) is significantly higher than the Median (8,052). This "Positive Skew" (0.71) mathematically confirms that the state's average is being "pulled up" by a few high-performing districts (like West and North West Delhi), masking lower performance in others.

Performance Classification:

78.6% (11 districts) are in the "Excellent" category, exceeding the excellence threshold.

21.4% (3 districts) are lagging, with Najafgarh and North East * falling into the "Critical" zone (<100 total updates).

ROI Metric: The system estimates an intervention cost of ₹6.0 Lakhs to train and stabilize the 3 underperforming districts, identifying a clear path for resource allocation.

2. Regional Inequality: The Gini Audit The Lorenz Curve and Gini Coefficient provide a research-level view of how Aadhaar services are distributed across Delhi.

Gini Coefficient (-0.45): While the interpretation is labeled as "LOW" inequality, the Lorenz Curve shows a specific concentration: the Bottom 50% of districts handle only 15.3% of the workload.

The Top Performer: West Delhi is the undisputed anchor of the system with 35,105 total updates, nearly 300 times the volume of South East Delhi (127 updates). This indicates a "Digital Service Desert" in the South East and Najafgarh regions.

3. Temporal Anomalies: System Volatility The Change Point Detection and Monthly Summaries identify specific months where the biometric system experienced massive shifts.

The July Spike: July recorded the highest activity (30,480 updates). In West Delhi specifically, July saw a surge to 10,704 updates (a Z-score of 2.48), likely due to post-school admission biometric requirements.

The January System Collapse: There is a severe "Crisis Drop" visible in the Change Point Detection graph for January 2026.

Total updates plummeted to 5,214 from a December high of 19,743.

New Delhi recorded a staggering 84.8% drop in January, with a Z-score of -0.46, representing a near-total cessation of biometric updates in the capital district.

The August Blackout: The data shows 0 updates in August, signaling a potential state-wide technical outage or a data reporting failure that requires immediate audit investigation.

4. District-Specific Audit (Priority Intervention List) Using Z-Score Anomaly Detection, we have identified the districts requiring urgent administrative attention:

Najafgarh & North East *: Flagged as Critical. These districts are consistently underperforming the state mean across all months.

South East Delhi: Despite being an active region, it has a total update count of only 127 over 68 active days, indicating extremely low daily throughput (1.86 mean daily updates).

New Delhi (Temporal Failure): While generally "Excellent," the district failed to maintain targets in October (45 updates) and January (10 updates), showing the highest volatility in the state.

Summary of Auditor Recommendations Immediate Action: Investigate the January Collapse in New Delhi and Central Delhi to determine if it was caused by Seva Kendra closures or software synchronization issues.

Strategic Investment: Allocate the ₹6.0L training budget specifically to Najafgarh and South East Delhi to bridge the "Service Gap."

Success Replication: Study the workflow of West Delhi, which maintains a high daily average (26.7 updates/day) even during low-performance months, to create a state-wide best-practice manual.

5 . TECHNICAL IMPLEMENTATION

5.1 Code Architecture

The project is built on a Modular Autonomous Pipeline designed for high-velocity data auditing. The architecture follows a strict "Zero-Touch" flow to ensure complete reproducibility.

Ingestion Layer: Utilizes Python’s requests and Pathlib to interface with APIs, pulling multi-gigabyte raw Aadhaar datasets directly into a structured Data/Raw directory.

Processing Layer (EDA & Cleaning): A dedicated module that standardizes schemas across 40 States/UTs, handles missing values, and exports cleaned CSVs for secondary analysis.

Mathematical Engine (Statistics): A research-grade module that performs univariate analysis, Gini Coefficient calculations, and Bayesian change point detection on the processed data.

Multimodal AI Orchestrator: The "Deep Analysis" bridge that encodes visual plots into Base64 and pairs them with statistical JSON summaries for high-fidelity LLM inference using meta-llama/llama-4-scout-17b.

Synthesis Layer (PDF): An automated reporting engine using ReportLab that assembles all saved insights and plots into a professional audit document without manual document formatting.

5.2 Key Functions

Our implementation relies on several proprietary functions designed for memory efficiency and deep analytical rigor:

generate_state_insights(state_name): The core orchestrator. It executes the multimodal loop—reading local JSON results, encoding visualizations, and calling the AI Auditor to produce categorical narratives for each state.

compute_univariate_statistics(series): A robust mathematical function that calculates Mean-Median gaps, Skewness, Kurtosis, and ROI metrics (training costs) to provide a financial dimension to the data audit.

bayesian_change_point_detection(series): Identifies critical "Crisis Drops" or "Sudden Improvements" by calculating posterior means and Z-scores across a rolling temporal window.

plot_matrix_anomalies(anomalies_df): A visualization utility that transforms raw Z-score data into a high-density "Urgent Intervention" heatmap, allowing for rapid geographic bottleneck identification.

discover_reports(): An autonomous file-mapping utility that scans the REPORTS_ROOT to dynamically build a state-folder map, ensuring the pipeline can scale to any number of states without hard-coded paths

6. CONCLUSIONS

6.1 Key Discoveries

Our research-level audit across 40 States and UTs has uncovered critical systemic patterns as of January 2026:

Systemic Volatility: The identification of the "January Collapse" in Delhi and the "August Blackout" across multiple datasets highlights significant vulnerabilities in reporting infrastructure that traditional averages failed to detect.

Regional Service Deserts: Mathematical modeling (Gini Coefficient & Lorenz Curves) revealed that despite high state-wide volumes, service delivery is often concentrated in top-tier districts, leaving regions like Najafgarh and South East Delhi in a "Critical" underperformance state.

Targeted ROI Opportunities: The pipeline successfully mapped underperformance to specific financial intervention targets, identifying a clear path for resource allocation (e.g., the ₹6.0L training budget for Delhi's lagging districts).

6.2 Impact Potential

The implementation of this autonomous AI Auditor offers transformative potential for UIDAI governance:

Rapid Crisis Response: By automating Bayesian change point detection, authorities can identify and investigate system crashes within days rather than waiting for quarterly reviews.

Data-Driven Fiscal Policy: The integration of ROI and payback period math into Aadhaar auditing allows for a move toward "Precision Budgeting," where training funds are directed only where they maximize social utility.

Universal Parity: Continuous monitoring of district-level inequalities ensures that digital exclusion is mathematically tracked and administratively addressed to achieve 100% Aadhaar saturation.

6.3 Innovation Highlights

The project successfully demonstrated a paradigm shift in data science:

End-to-End Autonomy: We moved from manual data cleaning and document formatting to a "Zero-Touch" modular pipeline that handles everything from API ingestion to final PDF synthesis.

Multimodal Analytical Depth: Our proprietary bridge between statistical JSON results and Base64 visual embeddings allowed the AI to "see" and "think" about the data simultaneously, producing insights that are both data-grounded and visually verified.

Reproducible Research Architecture: The entire logic exists in a modular, memory-efficient framework that can be instantly redeployed for any future UIDAI audit cycle with zero manual reconfiguration

8. APPENDIX: FULL CODE

8.1 Statistics Module

import numpy as np import pandas as pd import os import json from statsmodels.tsa.seasonal import STL import matplotlib.pyplot as plt import seaborn as sns from pathlib import Path import gc # For memory management

Set style for visually appealing plots

plt.style.use('seaborn-v0_8') sns.set_palette("husl")

==========

POLICY THRESHOLDS

==========

POLICY_THRESHOLDS = { 'critical_threshold': 100, # Below this = crisis 'target_threshold': 500, # Minimum acceptable 'excellence_threshold': 1000, # High performance 'cost_per_training': 200000, # ₹2l per district training 'value_per_update': 1000, # ₹1000 per enrollment value 'monthly_target': 50, # Target updates per district per month }

def compute_univariate_statistics(series: pd.Series) -> dict: """ Compute classical and robust univariate statistics WITH POLICY CONTEXT. """ series = series.dropna()

```
    if len(series) == 0:
        return {}

    mean = series.mean()
    median = series.median()
    variance = series.var(ddof=1)
    std_dev = series.std(ddof=1)

    q1 = series.quantile(0.25)
    q3 = series.quantile(0.75)
    iqr = q3 - q1

    mad = np.median(np.abs(series - median))
    skewness = series.skew()
    excess_kurtosis = series.kurt()

    # POLICY-RELEVANT ADDITIONS
    p10 = series.quantile(0.10)
    p90 = series.quantile(0.90)
    p95 = series.quantile(0.95)

    # Performance categories
    critical_count = (series < POLICY_THRESHOLDS['critical_threshold']).sum()
    below_target_count = ((series >= POLICY_THRESHOLDS['critical_threshold']) &
                          (series < POLICY_THRESHOLDS['target_threshold'])).sum()
    on_track_count = ((series >= POLICY_THRESHOLDS['target_threshold']) &
                      (series < POLICY_THRESHOLDS['excellence_threshold'])).sum()
    excellent_count = (series >= POLICY_THRESHOLDS['excellence_threshold']).sum()

    total_count = series.count()

    # Inequality measure (Gini coefficient)
    def gini_coefficient(x):
        sorted_x = np.sort(x)
        n = len(x)
        cumsum = np.cumsum(sorted_x)
        if cumsum[-1] == 0:
            return 0
        return (2 * np.sum((n - np.arange(1, n+1) + 1) * sorted_x)) / (n *
    cumsum[-1]) - 1

    gini = gini_coefficient(series.values)
```

```
# Concentration: what % do top 10% handle?
sorted_series = series.sort_values(ascending=False)
top_10_pct_idx = max(1, int(len(sorted_series) * 0.1))
total_sum = sorted_series.sum()
top_10_pct_contribution = sorted_series.head(top_10_pct_idx).sum() /
total_sum * 100 if total_sum > 0 else 0

bottom_50_pct_idx = int(len(sorted_series) * 0.5)
bottom_50_pct_contribution = sorted_series.tail(bottom_50_pct_idx).sum() /
total_sum * 100 if total_sum > 0 else 0

return {
    # Traditional statistics
    "mean": float(mean),
    "median": float(median),
    "variance": float(variance),
    "std_dev": float(std_dev),
    "iqr": float(iqr),
    "mad": float(mad),
    "skewness": float(skewness),
    "excess_kurtosis": float(excess_kurtosis),
    "min": float(series.min()),
    "max": float(series.max()),
    "count": int(total_count),

    # Policy-relevant percentiles
    "p10_bottom_10pct": float(p10),
    "q1_25th_percentile": float(q1),
    "q3_75th_percentile": float(q3),
    "p90_top_10pct": float(p90),
    "p95_top_5pct": float(p95),

    # Performance categories
    "critical_count": int(critical_count),
    "critical_percentage": float(critical_count / total_count * 100),
    "below_target_count": int(below_target_count),
    "below_target_percentage": float(below_target_count / total_count * 100),
    "on_track_count": int(on_track_count),
    "on_track_percentage": float(on_track_count / total_count * 100),
    "excellent_count": int(excellent_count),
    "excellent_percentage": float(excellent_count / total_count * 100),

    # Inequality measures
    "gini_coefficient": float(gini),
    "inequality_interpretation": "HIGH" if gini > 0.6 else "MODERATE" if gini
> 0.4 else "LOW",
    "top_10pct_contribution": float(top_10_pct_contribution),
    "bottom_50pct_contribution": float(bottom_50_pct_contribution),

    # Gap analysis
    "median_mean_gap": float(mean - median),
    "gap_interpretation": "High inequality - few outliers driving average up"
```

```
    if mean > 2 * median else "Moderate inequality",

        # Intervention estimates
        "districts_needing_intervention": int(critical_count +
    below_target_count),
        "intervention_percentage": float((critical_count + below_target_count) /
    total_count * 100),
        "estimated_training_cost_lakhs": float((critical_count +
    below_target_count) * POLICY_THRESHOLDS['cost_per_training'] / 100000),
    }
```

def compute_stats_for_dataframe(df: pd.DataFrame, columns: list[str]) -> pd.DataFrame: """Compute univariate statistics for multiple columns.""" results = {} for col in columns: results[col] = compute_univariate_statistics(df[col]) return pd.DataFrame(results).T

==========

DISTRICT-LEVEL AGGREGATION AND ANALYSIS

==========

def aggregate_by_district(df: pd.DataFrame, district_col: str, value_col: str) -> pd.DataFrame: """ Aggregate data by district with performance categorization. Returns district-level summary with names and categories. """ district_agg = df.groupby(district_col).agg({ value_col: ['sum', 'mean', 'median', 'std', 'count'] }).reset_index()

```
    district_agg.columns = [district_col, 'total_updates', 'mean_daily_updates',
                            'median_daily_updates', 'std_updates', 'days_active']

    # Add performance category
    def categorize_performance(total):
        if total < POLICY_THRESHOLDS['critical_threshold']:
            return 'Critical'
        elif total < POLICY_THRESHOLDS['target_threshold']:
            return 'Below Target'
        elif total < POLICY_THRESHOLDS['excellence_threshold']:
            return 'On Track'
        else:
            return 'Excellent'

    district_agg['performance_category'] =
    district_agg['total_updates'].apply(categorize_performance)
    district_agg['needs_intervention'] =
    district_agg['performance_category'].isin(['Critical', 'Below Target'])
```

```
    return district_agg.sort_values('total_updates', ascending=False)
```

def get_district_month_summary(df: pd.DataFrame, district_col: str, date_col: str, value_col: str) -> pd.DataFrame: """ Get month-wise summary for each district with anomaly flags. """ df_copy = df.copy() df_copy['month'] = pd.to_datetime(df_copy[date_col]).dt.strftime('%B') df_copy['year_month'] = pd.to_datetime(df_copy[date_col]).dt.to_period('M')

```
    monthly_summary = df_copy.groupby([district_col, 'month',
    'year_month']).agg({
        value_col: 'sum'
    }).reset_index()

    monthly_summary.columns = [district_col, 'month', 'year_month',
    'monthly_updates']

    # Add performance flags
    monthly_summary['meets_target'] = monthly_summary['monthly_updates'] >=
    POLICY_THRESHOLDS['monthly_target']
    monthly_summary['critical'] = monthly_summary['monthly_updates'] <
    POLICY_THRESHOLDS['critical_threshold']

    # Calculate month-over-month change for anomaly detection
    monthly_summary = monthly_summary.sort_values([district_col, 'year_month'])
    monthly_summary['pct_change'] = monthly_summary.groupby(district_col)
    ['monthly_updates'].pct_change()
    monthly_summary['concerning_drop'] = monthly_summary['pct_change'] < -0.3  #
    30% drop
    monthly_summary['sudden_spike'] = monthly_summary['pct_change'] > 1.0  # 100%
    increase

    return monthly_summary
```

==========

TEMPORAL STATISTICS

==========

def perform_stl_decomposition(df: pd.DataFrame, date_col: str, value_col: str, period: int = 12) -> pd.DataFrame: """STL decomposition with policy interpretation.""" if df.empty: raise ValueError("Input DataFrame is empty.")

```
    ts_df = df[[date_col, value_col]].copy()
    ts_df[date_col] = pd.to_datetime(ts_df[date_col])
```

```
    ts_df = ts_df.sort_values(date_col).set_index(date_col)

    # Resample to monthly frequency
    ts_df = ts_df.resample('M').sum()

    if len(ts_df) < period:
        raise ValueError(f"Not enough data for STL decomposition. Need at least
    {period} months.")

    stl = STL(ts_df[value_col], period=period, robust=True)
    result = stl.fit()

    out = ts_df.copy()
    out["trend"] = result.trend
    out["seasonal"] = result.seasonal
    out["residual"] = result.resid

    # Policy additions
    out['trend_direction'] = 'stable'
    out.loc[out['trend'].diff() > out['trend'].std() * 0.5, 'trend_direction'] =
    'increasing'
    out.loc[out['trend'].diff() < -out['trend'].std() * 0.5, 'trend_direction'] =
    'decreasing'

    out['meets_monthly_target'] = out[value_col] >=
    POLICY_THRESHOLDS['monthly_target']
    out['concerning_drop'] = out[value_col].pct_change() < -0.3

    out.reset_index(inplace=True)
    out.rename(columns={date_col: 'month'}, inplace=True)
    out['month'] = out['month'].dt.strftime('%B')

    return out
```

def bayesian_change_point_detection(series: pd.Series, window: int = 3, threshold: float = 3.0) -> pd.DataFrame: """Bayesian-style change point detection with policy interpretation.""" if series.empty: raise ValueError("Input series is empty.")

```
    monthly_series = series.resample('M').sum()
    df = pd.DataFrame({"value": monthly_series}).copy()

    df["posterior_mean"] = df["value"].rolling(window).mean()
    df["posterior_std"] = df["value"].rolling(window).std()
    df["z_score"] = (df["value"] - df["posterior_mean"]) / df["posterior_std"]
    df["change_point"] = df["z_score"].abs() > threshold

    # Policy additions
    df['change_type'] = 'normal'
    df.loc[(df['change_point']) & (df['z_score'] < -threshold), 'change_type'] =
    'crisis_drop'
    df.loc[(df['change_point']) & (df['z_score'] > threshold), 'change_type'] =
```

```
    'sudden_improvement'

    df['requires_investigation'] = df['change_type'].isin(['crisis_drop',
    'sudden_improvement'])
    df['policy_note'] = ''
    df.loc[df['change_type'] == 'crisis_drop', 'policy_note'] = 'Investigate:
    What caused this drop?'
    df.loc[df['change_type'] == 'sudden_improvement', 'policy_note'] = 'Document
    success: What intervention worked?'

    df.reset_index(inplace=True)
    df.rename(columns={series.index.name or 'date': 'month'}, inplace=True)
    df['month'] = df['month'].dt.strftime('%B')

    return df
```

def build_district_month_matrix(df: pd.DataFrame, district_col: str, date_col: str, value_col: str) -> pd.DataFrame: """Creates a District × Month matrix."""" data = df.copy() data[date_col] = pd.to_datetime(data[date_col]) data["month"] = data[date_col].dt.strftime('%B')

```
    matrix = data.groupby([district_col, "month"])
    [value_col].sum().unstack(fill_value=0)
    return matrix
```

def compute_matrix_stats(matrix: pd.DataFrame) -> pd.DataFrame: """Computes row/column level statistics with policy interpretation."""" stats = pd.DataFrame({ "row_sum_total_updates": matrix.sum(axis=1), "row_mean_monthly_updates": matrix.mean(axis=1), "row_variance": matrix.var(axis=1), "row_min": matrix.min(axis=1), "row_max": matrix.max(axis=1), })

```
    # Policy additions
    stats['performance_category'] = 'below_target'
    stats.loc[stats['row_mean_monthly_updates'] >=
    POLICY_THRESHOLDS['target_threshold'], 'performance_category'] = 'on_track'
    stats.loc[stats['row_mean_monthly_updates'] >=
    POLICY_THRESHOLDS['excellence_threshold'], 'performance_category'] =
    'excellent'
    stats.loc[stats['row_mean_monthly_updates'] <
    POLICY_THRESHOLDS['critical_threshold'], 'performance_category'] = 'critical'

    stats['needs_intervention'] = stats['performance_category'].isin(['critical',
    'below_target'])
    stats['consistency_score'] = 1 / (1 + stats['row_variance'])
    stats['months_below_target'] = (matrix <
    POLICY_THRESHOLDS['monthly_target']).sum(axis=1)

    return stats
```

def compute_zscore_anomalies(matrix: pd.DataFrame, z_thresh: float = 3.0) -> pd.DataFrame: """Detects abnormal district-months using Z-score with policy interpretation."""" mean = matrix.mean(axis=1) std =

matrix.std(axis=1)

```
    z_scores = matrix.sub(mean, axis=0).div(std, axis=0)

    anomalies = z_scores.stack().reset_index().rename(columns={0: "z_score"})
    anomalies["is_anomaly"] = anomalies["z_score"].abs() > z_thresh

    # Policy additions
    anomalies['anomaly_type'] = 'normal'
    anomalies.loc[(anomalies['is_anomaly']) & (anomalies['z_score'] < -z_thresh),
    'anomaly_type'] = 'underperformance'
    anomalies.loc[(anomalies['is_anomaly']) & (anomalies['z_score'] > z_thresh),
    'anomaly_type'] = 'exceptional_performance'

    anomalies['action_required'] = ''
    anomalies.loc[anomalies['anomaly_type'] == 'underperformance',
    'action_required'] = 'Investigate root cause'
    anomalies.loc[anomalies['anomaly_type'] == 'exceptional_performance',
    'action_required'] = 'Document best practices'

    return anomalies
```

==========

POLICY INSIGHT GENERATION

==========

def generate_policy_insights(stats_dict: dict, state: str, district_agg: pd.DataFrame, monthly_summary: pd.DataFrame, anomalies: pd.DataFrame) -> dict: """Generate actionable policy insights with district names and month-wise details.""" insights = { 'state': state, 'executive_summary': [], 'priority_actions': [], 'critical_districts': [], 'success_stories': [], 'month_wise_anomalies': [], 'risk_alerts': [], 'resource_allocation': {} }

```
    # Extract univariate stats
    uni_stats = stats_dict.get('univariate_stats', {}).get('total_updates', {})

    # EXECUTIVE SUMMARY
    if uni_stats:
        insights['executive_summary'] = [
            f" PERFORMANCE OVERVIEW:",
            f"- {uni_stats.get('critical_percentage', 0):.1f}% of districts in
    CRITICAL state (< {POLICY_THRESHOLDS['critical_threshold']} updates)",
            f"- {uni_stats.get('below_target_percentage', 0):.1f}% below target
    performance",
            f"- {uni_stats.get('excellent_percentage', 0):.1f}% achieving
```

```
excellence",
        f"",
        f" INEQUALITY ANALYSIS:",
        f"- Gini coefficient: {uni_stats.get('gini_coefficient', 0):.2f}
({uni_stats.get('inequality_interpretation', 'N/A')})",
        f"- Top 10% of districts handle
{uni_stats.get('top_10pct_contribution', 0):.1f}% of all updates",
        f"- Bottom 50% contribute only
{uni_stats.get('bottom_50pct_contribution', 0):.1f}%",
        f"",
        f" INTERVENTION ESTIMATE:",
        f"- Districts needing intervention:
{uni_stats.get('districts_needing_intervention', 0)}",
        f"- Estimated training cost: ₹
{uni_stats.get('estimated_training_cost_lakhs', 0):.1f}L",
    ]

# CRITICAL DISTRICTS with names
critical_districts = district_agg[district_agg['performance_category'] ==
'Critical'].head(20)
for idx, row in critical_districts.iterrows():
    insights['critical_districts'].append({
        'district_name': row[district_agg.columns[0]],
        'total_updates': int(row['total_updates']),
        'average_daily': float(row['mean_daily_updates']),
        'performance_gap': float(POLICY_THRESHOLDS['critical_threshold'] -
row['total_updates']),
        'priority': 'URGENT' if row['total_updates'] < 10 else 'HIGH'
    })

# MONTH-WISE ANOMALIES with district names
crisis_anomalies = anomalies[anomalies['anomaly_type'] ==
'underperformance'].nsmallest(15, 'z_score')
for idx, row in crisis_anomalies.iterrows():
    insights['month_wise_anomalies'].append({
        'district_name': row[anomalies.columns[0]],
        'month': row[anomalies.columns[1]],
        'z_score': float(row['z_score']),
        'severity': 'SEVERE' if row['z_score'] < -5 else 'MODERATE',
        'action': row['action_required']
    })

# SUCCESS STORIES
excellent_districts = district_agg[district_agg['performance_category'] ==
'Excellent'].head(10)
for idx, row in excellent_districts.iterrows():
    insights['success_stories'].append({
        'district_name': row[district_agg.columns[0]],
        'total_updates': int(row['total_updates']),
        'consistency': 'High' if row.get('std_updates', 0) <
row['mean_daily_updates'] else 'Moderate',
        'action': 'Study and replicate best practices'
```

```
        })

    # PRIORITY ACTIONS
    if uni_stats.get('critical_percentage', 0) > 20:
        insights['priority_actions'].append({
            'priority': 'HIGH',
            'action': 'Immediate intervention in critical districts',
            'districts_affected': uni_stats.get('critical_count', 0),
            'estimated_cost': f"₹{uni_stats.get('critical_count', 0) *
    POLICY_THRESHOLDS['cost_per_training'] / 100000:.1f}L",
            'timeline': '30 days',
            'expected_outcome': 'Bring critical districts to minimum threshold'
        })

    # RESOURCE ALLOCATION - IMPROVED ROI MODEL
    num_districts_need_help = uni_stats.get('districts_needing_intervention', 0)
    training_cost = num_districts_need_help *
    POLICY_THRESHOLDS['cost_per_training']
    expected_monthly_improvement = POLICY_THRESHOLDS.get('expected_improvement',
    150)
    value_per_update = POLICY_THRESHOLDS['value_per_update']
```

Calculate expected benefit over 12 months

```
    monthly_benefit = num_districts_need_help * expected_monthly_improvement *
    value_per_update
    annual_benefit = monthly_benefit * 12
    roi = ((annual_benefit - training_cost) / training_cost * 100) if
    training_cost > 0 else 0
    payback_months = (training_cost / monthly_benefit) if monthly_benefit > 0
    else 999

    insights['resource_allocation'] = {
    'total_budget_required_lakhs': training_cost / 100000,
    'priority_districts': num_districts_need_help,
    'cost_per_district_lakhs': POLICY_THRESHOLDS['cost_per_training'] / 100000,
    'expected_monthly_improvement': expected_monthly_improvement,
    'monthly_benefit_lakhs': monthly_benefit / 100000,
    'annual_benefit_lakhs': annual_benefit / 100000,
    'roi_percentage': round(roi, 1),
    'payback_period_months': round(payback_months, 1),
    'net_benefit_year1_lakhs': (annual_benefit - training_cost) / 100000,
    'investment_viable': 'YES' if roi > 50 else 'MARGINAL' if roi > 0 else 'NO'

}
```

==========

ENHANCED PLOTTING FUNCTIONS

==========

def plot_district_performance_dashboard(district_agg: pd.DataFrame, output_dir: Path): """Create comprehensive district performance dashboard."""" fig = plt.figure(figsize=(18, 12)) gs = fig.add_gridspec(3, 2, hspace=0.3, wspace=0.3)

```
    district_col = district_agg.columns[0]

    # Top 15 performers
    ax1 = fig.add_subplot(gs[0, 0])
    top_15 = district_agg.head(15)
    colors = ['darkgreen' if cat == 'Excellent' else 'lightgreen' if cat == 'On
    Track'
              else 'orange' if cat == 'Below Target' else 'red'
              for cat in top_15['performance_category']]

    ax1.barh(range(len(top_15)), top_15['total_updates'], color=colors,
    edgecolor='black')
    ax1.set_yticks(range(len(top_15)))
    ax1.set_yticklabels(top_15[district_col], fontsize=9)
    ax1.set_xlabel('Total Updates', fontweight='bold')
    ax1.set_title('Top 15 Districts by Performance', fontsize=12,
    fontweight='bold')
    ax1.axvline(x=POLICY_THRESHOLDS['target_threshold'], color='orange',
    linestyle='--', label='Target')
    ax1.axvline(x=POLICY_THRESHOLDS['excellence_threshold'], color='green',
    linestyle='--', label='Excellence')
    ax1.legend()
    ax1.grid(True, alpha=0.3, axis='x')



    # Performance category distribution
    ax3 = fig.add_subplot(gs[1, 0])
    category_counts = district_agg['performance_category'].value_counts()
    colors_pie = ['red', 'darkgreen', 'lightgreen', 'orange']
    explode = [0.1 if cat == 'Critical' else 0 for cat in category_counts.index]

    wedges, texts, autotexts = ax3.pie(category_counts.values,
    labels=category_counts.index,
                                       autopct='%1.1f%%', colors=colors_pie,
    explode=explode,
                                       startangle=90, textprops={'fontsize': 10,
    'fontweight': 'bold'})
    ax3.set_title('District Distribution by Performance', fontsize=12,
    fontweight='bold')
```

```
# Intervention cost analysis
ax4 = fig.add_subplot(gs[1, 1])
intervention_needed = district_agg[district_agg['needs_intervention']]
cost_per_district = POLICY_THRESHOLDS['cost_per_training'] / 100000  # in
lakhs

categories = ['Critical\n(Urgent)', 'Below Target\n(High Priority)',
'Total\nIntervention Needed']
critical_count = len(district_agg[district_agg['performance_category'] ==
'Critical'])
below_target_count = len(district_agg[district_agg['performance_category'] ==
'Below Target'])
costs = [critical_count * cost_per_district,
         below_target_count * cost_per_district,
         len(intervention_needed) * cost_per_district]

bars = ax4.bar(categories, costs, color=['red', 'orange', 'purple'],
edgecolor='black', linewidth=1.5)
ax4.set_ylabel('Cost (₹ Lakhs)', fontweight='bold')
ax4.set_title('Training Investment Required', fontsize=12, fontweight='bold')

for bar, cost, count in zip(bars, costs, [critical_count, below_target_count,
len(intervention_needed)]):
    height = bar.get_height()
    ax4.text(bar.get_x() + bar.get_width()/2., height,
            f'₹{cost:.1f}L\n({count} districts)',
            ha='center', va='bottom', fontweight='bold', fontsize=10)

# Geographic spread (if possible)
ax5 = fig.add_subplot(gs[2, :])

# Scatter: Total updates vs Consistency
scatter_colors = ['red' if cat == 'Critical' else 'orange' if cat == 'Below
Target'
                  else 'lightgreen' if cat == 'On Track' else 'darkgreen'
                  for cat in district_agg['performance_category']]

ax5.scatter(district_agg['total_updates'],
district_agg['mean_daily_updates'],
           c=scatter_colors, s=100, alpha=0.6, edgecolors='black',
linewidths=0.5)

# Add labels for extreme cases
for idx, row in district_agg.head(5).iterrows():
    ax5.annotate(row[district_col],
                 xy=(row['total_updates'], row['mean_daily_updates']),
                 xytext=(10, 10), textcoords='offset points',
                 fontsize=8, fontweight='bold',
                 bbox=dict(boxstyle='round,pad=0.3', facecolor='yellow',
alpha=0.7))

for idx, row in district_agg.tail(5).iterrows():
```

```
        ax5.annotate(row[district_col],
                     xy=(row['total_updates'], row['mean_daily_updates']),
                     xytext=(10, -10), textcoords='offset points',
                     fontsize=8, fontweight='bold', color='red',
                     bbox=dict(boxstyle='round,pad=0.3', facecolor='pink',
    alpha=0.7))

    ax5.set_xlabel('Total Updates (Cumulative)', fontweight='bold')
    ax5.set_ylabel('Average Daily Updates', fontweight='bold')
    ax5.set_title('District Performance Map: Total vs Daily Average',
    fontsize=12, fontweight='bold')
    ax5.grid(True, alpha=0.3)

    # Add threshold lines
    ax5.axvline(x=POLICY_THRESHOLDS['critical_threshold'], color='red',
    linestyle='--', alpha=0.5, label='Crisis')
    ax5.axvline(x=POLICY_THRESHOLDS['target_threshold'], color='orange',
    linestyle='--', alpha=0.5, label='Target')
    ax5.axvline(x=POLICY_THRESHOLDS['excellence_threshold'], color='green',
    linestyle='--', alpha=0.5, label='Excellence')
    ax5.legend()

    plt.suptitle('DISTRICT PERFORMANCE DASHBOARD - ACTIONABLE INSIGHTS',
                 fontsize=16, fontweight='bold', y=0.98)

    plt.savefig(output_dir / 'district_performance_dashboard.png', dpi=300,
    bbox_inches='tight')
    plt.close()

    # Clear memory
    gc.collect()
```

def plot_month_wise_heatmap(monthly_summary: pd.DataFrame, output_dir: Path): """Create month-wise heatmap with district names.""" district_col = monthly_summary.columns[0]

```
    # Grouping to combine different years into a single month entry
    seasonal_summary = monthly_summary.groupby('month')
    ['monthly_updates'].sum().reset_index()

    # Pivot for heatmap
    pivot_data = monthly_summary.pivot_table(
        index=district_col,
        columns='month',
        values='monthly_updates',
        aggfunc='sum'
    ).fillna(0)

    # Sort districts by total performance
    pivot_data['total'] = pivot_data.sum(axis=1)
    pivot_data = pivot_data.sort_values('total', ascending=False).drop('total',
    axis=1)
```

```
    # Limit to top 30 and bottom 15 for readability
    top_30 = pivot_data.head(30)
    bottom_15 = pivot_data.tail(15)
    display_data = pd.concat([top_30, bottom_15])

    # Create figure
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(16, 20))

    # Month order
    month_order = ['January', 'February', 'March', 'April', 'May', 'June',
                   'July', 'August', 'September', 'October', 'November',
    'December']
    display_data = display_data[[m for m in month_order if m in
    display_data.columns]]

    # Top performers heatmap
    sns.heatmap(top_30, annot=True, fmt='.0f', cmap='RdYlGn',
    center=POLICY_THRESHOLDS['monthly_target'],
                cbar_kws={'label': 'Monthly Updates'}, ax=ax1, linewidths=0.5,
    linecolor='gray')
    ax1.set_title('TOP 30 DISTRICTS - Monthly Performance Heatmap', fontsize=14,
    fontweight='bold')
    ax1.set_xlabel('Month', fontweight='bold')
    ax1.set_ylabel('District', fontweight='bold')

    # Crisis districts heatmap
    sns.heatmap(bottom_15, annot=True, fmt='.0f', cmap='Reds',
                cbar_kws={'label': 'Monthly Updates'}, ax=ax2, linewidths=0.5,
    linecolor='darkred')
    ax2.set_title('BOTTOM 15 DISTRICTS - Crisis Intervention Required',
    fontsize=14, fontweight='bold', color='red')
    ax2.set_xlabel('Month', fontweight='bold')
    ax2.set_ylabel('District', fontweight='bold')

    plt.tight_layout()
    plt.savefig(output_dir / 'month_wise_district_heatmap.png', dpi=300,
    bbox_inches='tight')
    plt.close()
    gc.collect()
```

def plot_univariate_stats(df: pd.DataFrame, columns: list[str], output_dir: Path): """Generate policy-oriented univariate plots.""" for col in columns: fig, axes = plt.subplots(2, 2, figsize=(14, 10)) fig.suptitle(f'Performance Analysis: {col}', fontsize=16, fontweight='bold') ax1 = axes[0, 0] data = df[col].dropna() # Histogram with zones n, bins, patches = ax1.hist(data, bins=50, edgecolor='black', alpha=0.7) for i, patch in enumerate(patches): bin_center = (bins[i] + bins[i+1]) / 2 if bin_center < POLICY_THRESHOLDS['critical_threshold']: patch.set_facecolor('red') elif bin_center < POLICY_THRESHOLDS['target_threshold']: patch.set_facecolor('orange') elif bin_center < POLICY_THRESHOLDS['excellence_threshold']: patch.set_facecolor('lightgreen') else: patch.set_facecolor('darkgreen') ax1.axvline(x=POLICY_THRESHOLDS['critical_threshold'], color='red',

linestyle='--', linewidth=2, label='Crisis') ax1.axvline(x=POLICY_THRESHOLDS['target_threshold'], color='orange', linestyle='--', linewidth=2, label='Target') ax1.axvline(x=POLICY_THRESHOLDS['excellence_threshold'], color='green', linestyle='--', linewidth=2, label='Excellence') ax1.set_title('Distribution by Performance Zones', fontweight='bold') ax1.set_xlabel(col) ax1.set_ylabel('Number of Districts') ax1.legend() # Performance categories ax2 = axes[0, 1] critical_count = (data < POLICY_THRESHOLDS['critical_threshold']).sum() below_target = ((data >= POLICY_THRESHOLDS['critical_threshold']) & (data < POLICY_THRESHOLDS['target_threshold'])).sum() on_track = ((data >= POLICY_THRESHOLDS['target_threshold']) & (data < POLICY_THRESHOLDS['excellence_threshold'])).sum() excellent = (data >= POLICY_THRESHOLDS['excellence_threshold']).sum() categories = ['Critical', 'Below Target', 'On Track', 'Excellent'] counts = [critical_count, below_target, on_track, excellent] colors_bar = ['red', 'orange', 'lightgreen', 'darkgreen'] bars = ax2.bar(categories, counts, color=colors_bar, edgecolor='black') ax2.set_title('Districts by Performance Category', fontweight='bold') ax2.set_ylabel('Number of Districts') total = len(data) for bar, count in zip(bars, counts): height = bar.get_height() ax2.text(bar.get_x() + bar.get_width()/2., height, f'{count}\n({count/total*100:.1f}%)', ha='center', va='bottom', fontweight='bold') # Lorenz curve ax3 = axes[1, 0] sorted_data = np.sort(data) cumsum = np.cumsum(sorted_data) cumsum_pct = cumsum / cumsum[-1] * 100 districts_pct = np.arange(1, len(sorted_data) + 1) / len(sorted_data) * 100 ax3.plot(districts_pct, cumsum_pct, linewidth=2, label='Actual Distribution') ax3.plot([0, 100], [0, 100], 'r--', linewidth=2, label='Perfect Equality') ax3.fill_between(districts_pct, cumsum_pct, districts_pct, alpha=0.3, color='orange') idx_50 = len(sorted_data) // 2 bottom_50_contribution = cumsum_pct[idx_50] ax3.annotate(f'Bottom 50% of districts\nhandle {bottom_50_contribution:.1f}% of work', xy=(50, bottom_50_contribution), xytext=(60, 30), arrowprops=dict(arrowstyle='->', color='red', lw=2), fontsize=10, color='red', fontweight='bold') ax3.set_title('Inequality Analysis (Lorenz Curve)', fontweight='bold') ax3.set_xlabel('Cumulative % of Districts') ax3.set_ylabel('Cumulative % of Updates') ax3.legend() ax3.grid(True, alpha=0.3) # Time series or percentiles ax4 = axes[1, 1] if isinstance(df.index, pd.DatetimeIndex): df[col].dropna().plot(ax=ax4, linewidth=2) ax4.axhline(y=POLICY_THRESHOLDS['monthly_target'], color='red', linestyle='--', linewidth=2, label='Monthly Target') ax4.set_title('Trend Over Time', fontweight='bold') ax4.legend() else: percentiles = data.quantile([0.1, 0.25, 0.5, 0.75, 0.9, 0.95]) labels = ['Bottom 10%', '25th %ile', 'Median', '75th %ile', '90th %ile', 'Top 5%'] bars = ax4.barh(range(len(percentiles)), percentiles.values, color='steelblue', edgecolor='black') ax4.set_yticks(range(len(percentiles))) ax4.set_yticklabels(labels) ax4.set_title('Percentile Distribution', fontweight='bold') ax4.set_xlabel(col) for i, (bar, val) in enumerate(zip(bars, percentiles.values)): ax4.text(val, i, f' {val:.0f}', va='center', fontweight='bold') plt.tight_layout() plt.savefig(output_dir / f'univariate_{col}_policy.png', dpi=300, bbox_inches='tight') plt.close() gc.collect()

def plot_stl_decomposition(stl_df: pd.DataFrame, month_col: str, value_col: str, output_dir: Path): """STL decomposition plot with duplicate month fix.""" fig, axes = plt.subplots(4, 1, figsize=(14, 12)) fig.suptitle('Temporal Decomposition: Trend, Seasonality & Anomalies', fontsize=16, fontweight='bold')

```
    # 1. FIX: Aggregate data by month to ensure each label appears only once
    # We take the mean for trend/seasonal/residual and sum for the raw values
    plot_df = stl_df.groupby(month_col).agg({
        value_col: 'sum',
```

```
    'trend': 'mean',
    'seasonal': 'mean',
    'residual': 'mean',
    'meets_monthly_target': 'max',  # If any district met target, mark as met
    'concerning_drop': 'max',
    'trend_direction': 'first'      # Keep the general trend label
}).reset_index()

# 2. Re-apply categorical ordering after aggregation
month_order = ['January', 'February', 'March', 'April', 'May', 'June',
               'July', 'August', 'September', 'October', 'November',
'December']
plot_df[month_col] = pd.Categorical(plot_df[month_col],
categories=month_order, ordered=True)
plot_df = plot_df.sort_values(month_col).reset_index(drop=True)

# 3. Use a clean range for x-axis positions
x_pos = np.arange(len(plot_df))

# --- Subplot 0: Original ---
axes[0].plot(x_pos, plot_df[value_col], marker='o', linewidth=2,
markersize=6)
axes[0].axhline(y=POLICY_THRESHOLDS['monthly_target'], color='red',
linestyle='--', linewidth=2, label='Monthly Target')

below_target = plot_df[plot_df[value_col] <
POLICY_THRESHOLDS['monthly_target']]
if not below_target.empty:
    axes[0].scatter(below_target.index, below_target[value_col], color='red',
s=100, zorder=5, label='Below Target')

axes[0].set_title('Original Series (Red dots = Below Target)',
fontweight='bold')

# --- Subplot 1: Trend ---
axes[1].plot(x_pos, plot_df['trend'], marker='s', linewidth=2, markersize=6)
for i, direction in enumerate(plot_df['trend_direction']):
    color = 'green' if direction == 'increasing' else 'red' if direction ==
'decreasing' else 'blue'
    if direction in ['increasing', 'decreasing']:
        axes[1].scatter(i, plot_df['trend'].iloc[i], color=color, s=100,
zorder=5)
axes[1].set_title('Trend (Green=Increasing, Red=Decreasing)',
fontweight='bold')

# --- Subplot 2: Seasonal ---
axes[2].plot(x_pos, plot_df['seasonal'], marker='^', linewidth=2,
markersize=6, color='purple')
axes[2].axhline(y=0, color='black', linestyle='-', linewidth=0.5)
axes[2].fill_between(x_pos, 0, plot_df['seasonal'], alpha=0.3,
color='purple')
axes[2].set_title('Seasonal Pattern', fontweight='bold')
```

```
    # --- Subplot 3: Residual ---
    axes[3].plot(x_pos, plot_df['residual'], marker='o', linewidth=2,
    markersize=6, color='gray')
    axes[3].axhline(y=0, color='black', linestyle='-', linewidth=0.5)

    concerning = plot_df[plot_df['concerning_drop'] == True]
    if not concerning.empty:
        axes[3].scatter(concerning.index, concerning['residual'], color='red',
    s=150, zorder=5,
                        marker='v', label='Concerning Drop')
    axes[3].set_title('Residual (Unexpected Variations)', fontweight='bold')

    # 4. Apply unique month labels to all subplots
    for ax in axes:
        ax.set_xticks(x_pos)
        ax.set_xticklabels(plot_df[month_col], rotation=45, ha='right')
        ax.grid(True, alpha=0.3)

    axes[0].legend()
    axes[3].legend()

    plt.tight_layout()
    plt.savefig(output_dir / 'stl_decomposition_policy.png', dpi=300,
    bbox_inches='tight')
    plt.close()
    gc.collect()
```

def plot_change_point_detection(cpd_df: pd.DataFrame, output_dir: Path): """Change point detection plot with duplicate month fix.""" fig, ax = plt.subplots(figsize=(14, 7))

```
    # 1. AGGREGATE: Collapse multiple rows per month into state-wide totals
    # We sum the values and take the 'max' for categorical/boolean flags
    plot_df = cpd_df.groupby('month').agg({
        'value': 'sum',
        'change_type': 'max',  # Prioritizes 'sudden_improvement' or
    'crisis_drop' over 'normal'
    }).reset_index()

    # 2. SORT: Ensure chronological order
    month_order = ['January', 'February', 'March', 'April', 'May', 'June',
                   'July', 'August', 'September', 'October', 'November',
    'December']
    plot_df['month'] = pd.Categorical(plot_df['month'], categories=month_order,
    ordered=True)
    plot_df = plot_df.sort_values('month').reset_index(drop=True)

    # 3. PLOT: Use unique positions
    x_pos = np.arange(len(plot_df))
    ax.plot(x_pos, plot_df['value'], linewidth=2, marker='o', markersize=6,
    label='Monthly Updates')
```

```
# Identify specific change points in the aggregated data
crisis_points = plot_df[plot_df['change_type'] == 'crisis_drop']
improvement_points = plot_df[plot_df['change_type'] == 'sudden_improvement']

if not crisis_points.empty:
    ax.scatter(crisis_points.index, crisis_points['value'], color='red',
s=200, zorder=5,
               marker='v', label='⚠️ Crisis Drop', edgecolors='darkred',
linewidths=2)

    for idx, row in crisis_points.iterrows():
        ax.annotate('INVESTIGATE', xy=(idx, row['value']), xytext=(idx,
row['value'] * 1.2),
                    arrowprops=dict(arrowstyle='->', color='red', lw=2),
                    fontsize=9, color='red', fontweight='bold', ha='center')

if not improvement_points.empty:
    ax.scatter(improvement_points.index, improvement_points['value'],
color='green', s=200, zorder=5,
               marker='^', label='✅ Success!', edgecolors='darkgreen',
linewidths=2)

    for idx, row in improvement_points.iterrows():
        ax.annotate('SUCCESS!', xy=(idx, row['value']), xytext=(idx,
row['value'] * 0.8),
                    arrowprops=dict(arrowstyle='->', color='green', lw=2),
                    fontsize=9, color='green', fontweight='bold', ha='center')

# Target line using POLICY_THRESHOLDS
ax.axhline(y=POLICY_THRESHOLDS['monthly_target'], color='orange',
linestyle='--',
          linewidth=2, label=f'Target
({POLICY_THRESHOLDS["monthly_target"]})')

ax.set_title('Change Point Detection: State-Wide Trends & Anomalies',
fontsize=14, fontweight='bold')
ax.set_xlabel('Month', fontweight='bold')
ax.set_ylabel('Total Updates', fontweight='bold')

# Apply unique month labels
ax.set_xticks(x_pos)
ax.set_xticklabels(plot_df['month'], rotation=45, ha='right')

ax.legend(loc='best', fontsize=10)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(output_dir / 'change_point_detection_policy.png', dpi=300,
bbox_inches='tight')
plt.close()
gc.collect()
```

def plot_matrix_anomalies(anomalies_df: pd.DataFrame, output_dir: Path): """Matrix anomalies heatmap with policy context.""" pivot = anomalies_df.pivot(index=anomalies_df.columns[0], columns=anomalies_df.columns[1], values='z_score')

```
    fig, ax = plt.subplots(figsize=(14, 10))
    sns.heatmap(pivot, annot=False, cmap='RdYlGn', center=0, ax=ax,
                cbar_kws={'label': 'Z-Score (Red=Crisis, Green=Excellence)'})

    ax.set_title('District-Month Performance Heatmap\n(Dark Red = Urgent
    Intervention Needed)',
                fontsize=14, fontweight='bold')
    ax.set_xlabel('Month', fontweight='bold')
    ax.set_ylabel('District', fontweight='bold')

    plt.tight_layout()
    plt.savefig(output_dir / 'matrix_anomalies_heatmap_policy.png', dpi=300,
    bbox_inches='tight')
    plt.close()

    crisis_anomalies = anomalies_df[anomalies_df['anomaly_type'] ==
    'underperformance']
    if not crisis_anomalies.empty:
        fig, ax = plt.subplots(figsize=(12, 6))

        top_crisis = crisis_anomalies.nsmallest(10, 'z_score')
        colors_crisis = ['red' if z < -5 else 'orange' for z in
    top_crisis['z_score']]

        ax.barh(range(len(top_crisis)), top_crisis['z_score'],
    color=colors_crisis, edgecolor='black')
        ax.set_yticks(range(len(top_crisis)))
        ax.set_yticklabels([f"{row[0]} - {row[1]}" for row in
    top_crisis[[top_crisis.columns[0], top_crisis.columns[1]]].values])
        ax.set_xlabel('Z-Score (More negative = Worse crisis)',
    fontweight='bold')
        ax.set_title('Top 10 Crisis District-Months (Priority Intervention
    List)', fontsize=14, fontweight='bold')
        ax.axvline(x=-3, color='red', linestyle='--', linewidth=2, label='Crisis
    Threshold')
        ax.legend()
        ax.grid(True, alpha=0.3, axis='x')

        plt.tight_layout()
        plt.savefig(output_dir / 'top_crisis_districts_policy.png', dpi=300,
    bbox_inches='tight')
        plt.close()

    gc.collect()
```

MAIN PROCESSING FUNCTION

==========

def process_statistics_for_state(state: str, data_dir: str = 'Data/Processed/biometric', reports_dir: str = 'Notebooks/Reports'): """Process statistics for a given state WITH POLICY INSIGHTS AND DISTRICT NAMES."""

```
    print(f"\n{'='*80}")
    print(f"PROCESSING POLICY STATISTICS FOR: {state.upper()}")
    print(f"{'='*80}\n")

    processed_csv_path = Path(data_dir) / f'{state}.csv'
    state_reports_dir = Path(reports_dir) / 'statistics_biometric' / state
    state_reports_dir.mkdir(parents=True, exist_ok=True)

    # Load data
    print(" Loading data...")
    df = pd.read_csv(processed_csv_path, parse_dates=['date'])
    df.set_index('date', inplace=True)

    print(f"   ✓ Loaded {len(df)} rows")
    print(f"   ✓ Date range: {df.index.min()} to {df.index.max()}")
    print(f"   ✓ Unique districts: {df['district'].nunique()}\n")

    district_col = 'district'
    date_col = 'date'
    value_col = 'total_updates'

    results = {}

    # STEP 1: Aggregate by district
    print(" Step 1: Aggregating data by district...")
    district_agg = aggregate_by_district(df, district_col, value_col)
    results['district_summary'] = district_agg.to_dict(orient='records')
    print(f"   ✓ {len(district_agg)} districts analyzed")
    print(f"   ✓ Critical:
    {len(district_agg[district_agg['performance_category']=='Critical'])}")
    print(f"   ✓ Needs intervention:
    {district_agg['needs_intervention'].sum()}\n")

    # STEP 2: Month-wise summary
    print(" Step 2: Creating month-wise district summary...")
    monthly_summary = get_district_month_summary(df.reset_index(), district_col,
    date_col, value_col)
    results['monthly_district_summary'] =
    monthly_summary.to_dict(orient='records')
    print(f"   ✓ {len(monthly_summary)} district-month combinations\n")
```

```
# STEP 3: Univariate statistics on AGGREGATED data
print(" Step 3: Computing univariate statistics...")
uni_stats = compute_stats_for_dataframe(district_agg, ['total_updates'])
results['univariate_stats'] = uni_stats.to_dict()
plot_univariate_stats(district_agg.set_index(district_col),
['total_updates'], state_reports_dir)
print(f"   ✓ Statistics computed\n")

# STEP 4: STL Decomposition
print(" Step 4: Performing STL decomposition...")
try:
    stl_df = perform_stl_decomposition(df.reset_index(), date_col, value_col,
period=12)
    results['stl_decomposition'] = stl_df.to_dict(orient='records')
    plot_stl_decomposition(stl_df, 'month', value_col, state_reports_dir)
    print(f"   ✓ STL decomposition completed\n")
except Exception as e:
    print(f"    STL decomposition skipped: {e}\n")

# STEP 5: Change Point Detection
print(" Step 5: Detecting change points...")
try:
    cpd_df = bayesian_change_point_detection(df[value_col])
    results['change_point_detection'] = cpd_df.to_dict(orient='records')
    plot_change_point_detection(cpd_df, state_reports_dir)
    print(f"   ✓ Change points detected\n")
except Exception as e:
    print(f"    Change point detection skipped: {e}\n")

# STEP 6: District-Month Matrix
print(" Step 6: Analyzing district-month patterns...")
matrix = build_district_month_matrix(df.reset_index(), district_col,
date_col, value_col)
matrix_stats = compute_matrix_stats(matrix)
anomalies = compute_zscore_anomalies(matrix)
results['matrix_stats'] = matrix_stats.to_dict()
results['matrix_anomalies'] = anomalies.to_dict(orient='records')
plot_matrix_anomalies(anomalies, state_reports_dir)
print(f"   ✓ Matrix analysis completed\n")

# STEP 7: Generate Policy Insights
print(" Step 7: Generating policy insights...")
policy_insights = generate_policy_insights(results, state, district_agg,
monthly_summary, anomalies)
results['policy_insights'] = policy_insights
print(f"   ✓ Policy insights generated\n")

# STEP 8: Create enhanced visualizations
print(" Step 8: Creating enhanced visualizations...")
plot_district_performance_dashboard(district_agg, state_reports_dir)
plot_month_wise_heatmap(monthly_summary, state_reports_dir)
print(f"   ✓ Visualizations created\n")
```

```
    # STEP 9: Save results
    print(" Step 9: Saving results...")
    with open(state_reports_dir / 'statistics_results.json', 'w') as f:
        json.dump(results, f, indent=4, default=str)


    print(f"   ✓ Results saved\n")

    print(f"{'='*80}")
    print(f" STATISTICS PROCESSING COMPLETED FOR {state.upper()}")
    print(f"{'='*80}\n")
    print(f" Output directory: {state_reports_dir}")
    print(f" Visualizations: {len(list(state_reports_dir.glob('*.png')))} images
    created\n")

    # Clear memory
    del df, district_agg, monthly_summary, matrix, matrix_stats, anomalies
    gc.collect()

    return results
```

==========

END OF FILE

==========

8.2 EDA Module

import pandas as pd import matplotlib.pyplot as plt import gc import seaborn as sns from pathlib import Path from config.api_config import RAW_DATA_DIR # Assuming this points to Data/Raw/ from utils.state_mapper import normalize_state_name

Set up seaborn for better plots

sns.set(style="whitegrid")

def feature_engineering(df: pd.DataFrame) -> pd.DataFrame: # Sum age group updates into total_updates df['total_updates'] = (df['age_0_5'] + df['age_5_17'] + df['age_18_greater']).astype('int32')

```
    # FIX: Added dayfirst=True to correctly parse DD-MM-YYYY and silence warning
    df['date'] = pd.to_datetime(df['date'], dayfirst=True, errors='coerce')
```

```
    # Drop rows where date is NaT
    df = df.dropna(subset=['date']).copy()

    # Extract month data
    df['month_number'] = df['date'].dt.month.astype('int8')
    df['month_name'] = df['date'].dt.month_name()

    return df
```

def perform_eda_and_plots(df: pd.DataFrame, state: str, output_dir: Path) -> str: """ Performs basic EDA and generates visualizations. Saves plots to output_dir and returns a text summary. """

```
    # Aggregate data for plots
    monthly_total = df.groupby('month_name')['total_updates'].sum().reset_index()
    monthly_age_groups = df.groupby('month_name')[['age_0_5',
    'age_5_17','age_18_greater']].sum().reset_index()
    district_total = df.groupby('district')
    ['total_updates'].sum().reset_index().sort_values('total_updates',
    ascending=False).head(20)  # Top 20 districts

    # Plot 1: Month name vs. total_updates (Bar plot)
    plt.figure(figsize=(10, 6))
    sns.barplot(data=monthly_total, x='month_name', y='total_updates',
    palette='viridis')
    plt.title(f'Total Updates by Month for {state}')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plot1_path = output_dir / f"{normalize_state_name(state)}_monthly_total.png"
    plt.savefig(plot1_path)
    plt.close()  # Free memory


    # Plot 2: Month name vs. updates by age groups (Stacked bar plot)
    plt.figure(figsize=(10, 6))
    monthly_age_groups.set_index('month_name').plot(kind='bar', stacked=True,
    figsize=(10, 6), colormap='Set2')
    plt.title(f'Updates by Age Groups and Month for {state}')
    plt.xticks(rotation=45)
    plt.ylabel('Updates')
    plt.tight_layout()
    plot2_path = output_dir / f"
    {normalize_state_name(state)}_monthly_age_groups.png"
    plt.savefig(plot2_path)
    plt.close()  # Free memory

    # Plot 3: District names vs. total_updates (Bar plot, top 20)
    plt.figure(figsize=(12, 8))
    sns.barplot(data=district_total, x='district', y='total_updates',
    palette='coolwarm')
    plt.title(f'Total Updates by District for {state} (Top 20)')
    plt.xticks(rotation=90)
```

```
    plt.tight_layout()
    plot3_path = output_dir / f"{normalize_state_name(state)}_district_total.png"
    plt.savefig(plot3_path)
    plt.close()  # Free memory
```

from config.api_config import RAW_DATA_DIR, PROCESSED_DATA_DIR, EDA_PLOTS_DIR # Add these to your config

def process_state_eda(state: str) -> str: normalized_state = normalize_state_name(state) raw_path = RAW_DATA_DIR / "enrolment" / f"{normalized_state}.csv" processed_dir = PROCESSED_DATA_DIR / "enrolment" processed_dir.mkdir(parents=True, exist_ok=True) plots_dir = EDA_PLOTS_DIR / normalized_state plots_dir.mkdir(parents=True, exist_ok=True)

```
    try:
        # Load raw CSV
        df = pd.read_csv(raw_path)

        if df.empty:
            return f" Warning: {state} dataset is empty."

        # Feature engineering
        df = feature_engineering(df)

        # Save processed CSV
        processed_path = processed_dir / f"{normalized_state}.csv"
        df.to_csv(processed_path, index=False)

        # Perform EDA and generate plots
        # Ensure perform_eda_and_plots returns a string summary!
        summary = perform_eda_and_plots(df, state, plots_dir)

        # Cleanup
        del df
        gc.collect()

        # FIX: Ensure a string is returned to avoid TypeError in the loop
        if not summary:
            summary = f"EDA completed for {state}. Plots saved to {plots_dir}."
        return summary

    except Exception as e:
        return f" Error processing {state}: {str(e)}"
```

Github Repository -- https://github.com/Kanishkasharma1908/UIDAI_Project-